Yolov11部署

1 YOLOv11推理(Python)

1.1 YOLOv11预测

在 YOLO11项目 主目录下新建 `predict.py` 预测文件，其内容如下：

```python
import cv2
from ultralytics import YOLO

def hsv2bgr(h, s, v):
    h_i = int(h * 6)
    f = h * 6 - h_i
    p = v * (1 - s)
    q = v * (1 - f * s)
    t = v * (1 - (1 - f) * s)

    r, g, b = 0, 0, 0

    if h_i == 0:
        r, g, b = v, t, p
    elif h_i == 1:
        r, g, b = q, v, p
    elif h_i == 2:
        r, g, b = p, v, t
    elif h_i == 3:
        r, g, b = p, q, v
    elif h_i == 4:
        r, g, b = t, p, v
    elif h_i == 5:
        r, g, b = v, p, q

    return int(b * 255), int(g * 255), int(r * 255)

def random_color(id):
    h_plane = (((id << 2) ^ 0x937151) % 100) / 100.0
    s_plane = (((id << 3) ^ 0x315793) % 100) / 100.0
    return hsv2bgr(h_plane, s_plane, 1)

if __name__ == "__main__":

    model = YOLO("/data/coding/ultralytics-main/runs/detect/train/weights/best.pt",task="detect")

```

```python
    img = cv2.imread("/data/coding/yolo_data/val/images/00002.jpg")
    results = model(img)[0]
    names   = results.names
    boxes   = results.boxes.data.tolist()

    for obj in boxes:
        left, top, right, bottom = int(obj[0]), int(obj[1]), int(obj[2]), int(obj[3])
        confidence = obj[4]
        label = int(obj[5])
        color = random_color(label)
        cv2.rectangle(img, (left, top), (right, bottom), color=color,thickness=2, lineType=cv2.LINE_AA)
        caption = f"{names[label]} {confidence:.2f}"
        w, h = cv2.getTextSize(caption, 0, 1, 2)[0]
        cv2.rectangle(img, (left - 3, top - 33), (left + w + 10, top), color, -1)
        cv2.putText(img, caption, (left, top - 5), 0, 1, (0, 0, 0), 2, 16)

    cv2.imwrite("predict.jpg", img)
    print("save done")
```

在终端执行文件`predict.py`,得到如下结果：

```
(torch) root@nblbofxhceaiffko-snow-764c6757db-z25mf:/data/coding/ultralytics-main# python predict.py

0: 480x640 1 Manhole, 102.8ms
Speed: 11.1ms preprocess, 102.8ms inference, 171.0ms postprocess per image at shape (1, 3, 480, 640)
save done
```

预测主要流程代码位于**ultralytics/engine/predictor.py** 文件中`BasePredictor`的`stream_inference`方法中：

```
ultralytics > engine > predictor.py
 67  class BasePredictor:
277      def stream_inference(self, source=None, model=None, *args, **kwargs):
298          with self._lock:  # for thread-safe inference
314              profilers = (
318              )
319              self.run_callbacks("on_predict_start")
320              for self.batch in self.dataset:
321                  self.run_callbacks("on_predict_batch_start")
322                  paths, im0s, s = self.batch
323
324                  # Preprocess预处理
325                  with profilers[0]:
326                      im = self.preprocess(im0s)
327
328                  # Inference推理
329                  with profilers[1]:
330                      preds = self.inference(im, *args, **kwargs)
331                      if self.args.embed:
332                          yield from [preds] if isinstance(preds, torch.Tensor) else preds  # yield embedding tensors
333                          continue
334
335                  # Postprocess后处理
336                  with profilers[2]:
337                      self.results = self.postprocess(preds, im, im0s)
338                  self.run_callbacks("on_predict_postprocess_end")
339
340                  # Visualize, save, write results
341                  n = len(im0s)
342                  try:
343                      for i in range(n):
344                          self.seen += 1
```

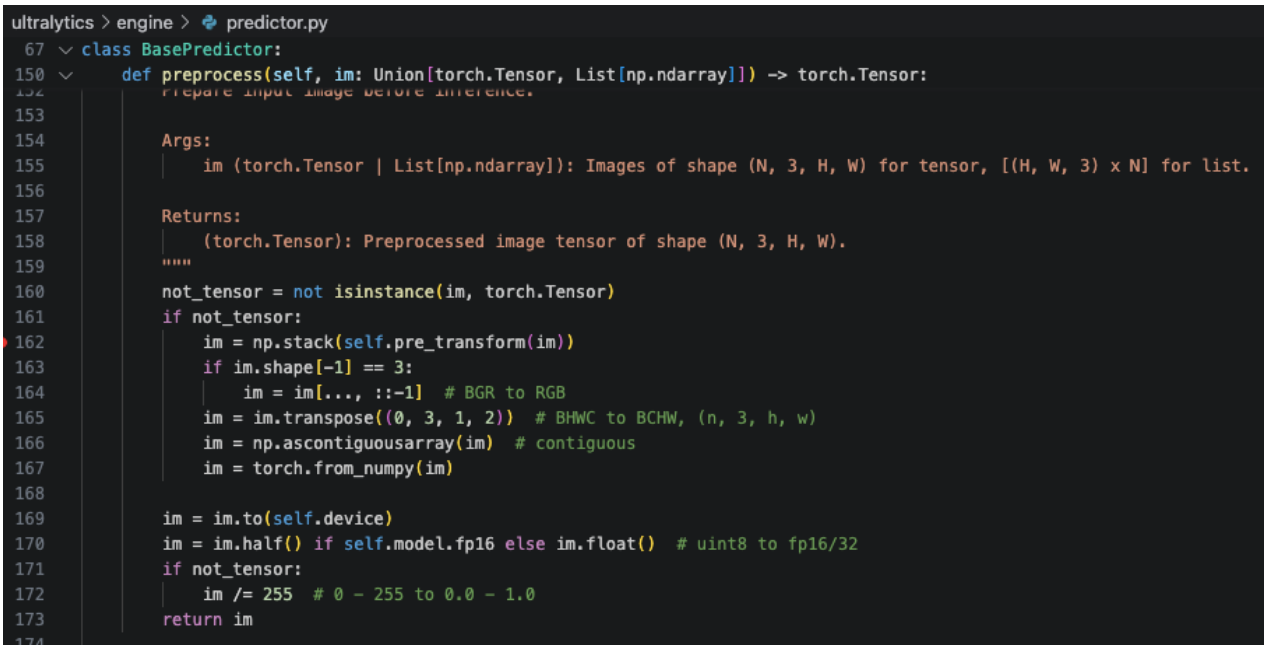
主要流程包括：

- self.preprocess：预处理

- self.inference:模型推理
- self.postprocess：后处理

1.2 YOLOv11预处理

通过调试分析可知 YOLO11 的预处理过程在 **ultralytics/engine/predictor.py** 文件中：



它包含以下步骤：

- self.pre_transform：即 letterbox 添加灰条、进行resize
- im[...,::-1]：BGR → RGB
- transpose((0, 3, 1, 2))：添加 batch 维度，HWC → CHW
- torch.from_numpy：to Tensor
- im /= 255：除以 255，归一化

其中：`letterbox`核心代码如下：

ultralytics > data > augment.py

```python
class LetterBox:
    def __call__(self, labels: Dict[str, Any] = None, image: np.ndarray = None) -> Union[Dict[str, Any], np.ndarray]:

        # Scale ratio (new / old) 计算resize的比例
        r = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
        if not self.scaleup:  # only scale down, do not scale up (for better val mAP)
            r = min(r, 1.0)

        # Compute padding
        ratio = r, r  # width, height ratios
        new_unpad = int(round(shape[1] * r)), int(round(shape[0] * r))#计算等比例缩放后的尺寸
        dw, dh = new_shape[1] - new_unpad[0], new_shape[0] - new_unpad[1]  # wh padding 计算全尺寸的pad量
        if self.auto:  # minimum rectangle
            dw, dh = np.mod(dw, self.stride), np.mod(dh, self.stride)  # wh padding 计算保证stride整除的pad量
        elif self.scale_fill:  # stretch
            dw, dh = 0.0, 0.0
            new_unpad = (new_shape[1], new_shape[0])
            ratio = new_shape[1] / shape[1], new_shape[0] / shape[0]  # width, height ratios

        if self.center:
            dw /= 2  # divide padding into 2 sides
            dh /= 2

        if shape[::-1] != new_unpad:  # resize
            img = cv2.resize(img, new_unpad, interpolation=cv2.INTER_LINEAR)#实现resize
            if img.ndim == 2:
                img = img[..., None]

        top, bottom = int(round(dh - 0.1)) if self.center else 0, int(round(dh + 0.1))
        left, right = int(round(dw - 0.1)) if self.center else 0, int(round(dw + 0.1))
        h, w, c = img.shape
        if c == 3:#进行灰度pad
            img = cv2.copyMakeBorder(img, top, bottom, left, right, cv2.BORDER_CONSTANT, value=(114, 114, 114))
        else:  # multispectral
            pad_img = np.full((h + top + bottom, w + left + right, c), fill_value=114, dtype=img.dtype)
            pad_img[top : top + h, left : left + w] = img
            img = pad_img
```

1.3 YOLOv11模型推理

在模型推理部分，首先会经过一个`AutoBackend`模块进行后端路由，根据不同的候选选择进行不同的模型推理：

```python
class AutoBackend(nn.Module):
    """
    Handle dynamic backend selection for running inference using Ultralytics YOLO models.

    The AutoBackend class is designed to provide an abstraction layer for various inference engines. It supports a wide
    range of formats, each with specific naming conventions as outlined below:

        Supported Formats and Naming Conventions:
            | Format                | File Suffix       |
            | --------------------- | ----------------- |
            | PyTorch               | *.pt              |
            | TorchScript           | *.torchscript     |
            | ONNX Runtime          | *.onnx            |
            | ONNX OpenCV DNN       | *.onnx (dnn=True) |
            | OpenVINO              | *openvino_model/  |
            | CoreML                | *.mlpackage       |
            | TensorRT              | *.engine          |
            | TensorFlow SavedModel | *_saved_model/    |
            | TensorFlow GraphDef   | *.pb              |
            | TensorFlow Lite       | *.tflite          |
            | TensorFlow Edge TPU   | *_edgetpu.tflite  |
            | PaddlePaddle          | *_paddle_model/   |
            | MNN                   | *.mnn             |
            | NCNN                  | *_ncnn_model/     |
            | IMX                   | *_imx_model/      |
            | RKNN                  | *_rknn_model/     |
```

ultralytics > nn > autobackend.py

```
70  class AutoBackend(nn.Module):
619      ) -> Union[torch.Tensor, List[torch.Tensor]]:
634          if self.fp16 and im.dtype != torch.float16:
635              im = im.half()  # to FP16
636          if self.nhwc:
637              im = im.permute(0, 2, 3, 1)  # torch BCHW to numpy BHWC shape(1,320,192,3)
638
639          # PyTorch
640          if self.pt or self.nn_module:
641              y = self.model(im, augment=augment, visualize=visualize, embed=embed, **kwargs)
642
643          # TorchScript
644          elif self.jit:
645              y = self.model(im)
646
647          # ONNX OpenCV DNN
648          elif self.dnn:
649              im = im.cpu().numpy()  # torch to numpy
650              self.net.setInput(im)
651              y = self.net.forward()
652
653          # ONNX Runtime
654          elif self.onnx or self.imx:
655              if self.dynamic:
656                  im = im.cpu().numpy()  # torch to numpy
657                  y = self.session.run(self.output_names, {self.session.get_inputs()[0].name: im})
658              else:
659                  if not self.cuda:
660                      im = im.cpu()
661                  self.io.bind_input(
662                      name="images",
663                      device_type=im.device.type,
664                      device_id=im.device.index if im.device.type == "cuda" else 0,
665                      element_type=np.float16 if self.fp16 else np.float32,
666                      shape=tuple(im.shape),
667                      buffer_ptr=im.data_ptr(),
668                  )
```

对于pytorch后端，最终跟训练时一样，由BaseModel的_predict_once方法负责，通过遍历网络的所有层进行前向传播推理：

ultralytics > nn > tasks.py

```
97  class BaseModel(torch.nn.Module):
159     def _predict_once(self, x, profile=False, visualize=False, embed=None):
161         Perform a forward pass through the network.
162
163         Args:
164             x (torch.Tensor): The input tensor to the model.
165             profile (bool): Print the computation time of each layer if True.
166             visualize (bool): Save the feature maps of the model if True.
167             embed (list, optional): A list of feature vectors/embeddings to return.
168
169         Returns:
170             (torch.Tensor): The last output of the model.
171         """
172         y, dt, embeddings = [], [], []  # outputs
173         embed = frozenset(embed) if embed is not None else {-1}
174         max_idx = max(embed)
175         for m in self.model:
176             if m.f != -1:  # if not from previous layer
177                 x = y[m.f] if isinstance(m.f, int) else [x if j == -1 else y[j] for j in m.f]  # from earlier layers
178             if profile:
179                 self._profile_one_layer(m, x, dt)
180             x = m(x)  # run
181             y.append(x if m.i in self.save else None)  # save output
182             if visualize:
183                 feature_visualization(x, m.type, m.i, save_dir=visualize)
184             if m.i in embed:
185                 embeddings.append(torch.nn.functional.adaptive_avg_pool2d(x, (1, 1)).squeeze(-1).squeeze(-1))  # flatten
186                 if m.i == max_idx:
187                     return torch.unbind(torch.cat(embeddings, 1), dim=0)
188         return x
```

但是，相对于训练流程，在最后一层`Detect`等的`forward`方法中增加了一步**解码**操作：

ultralytics > nn > modules > head.py

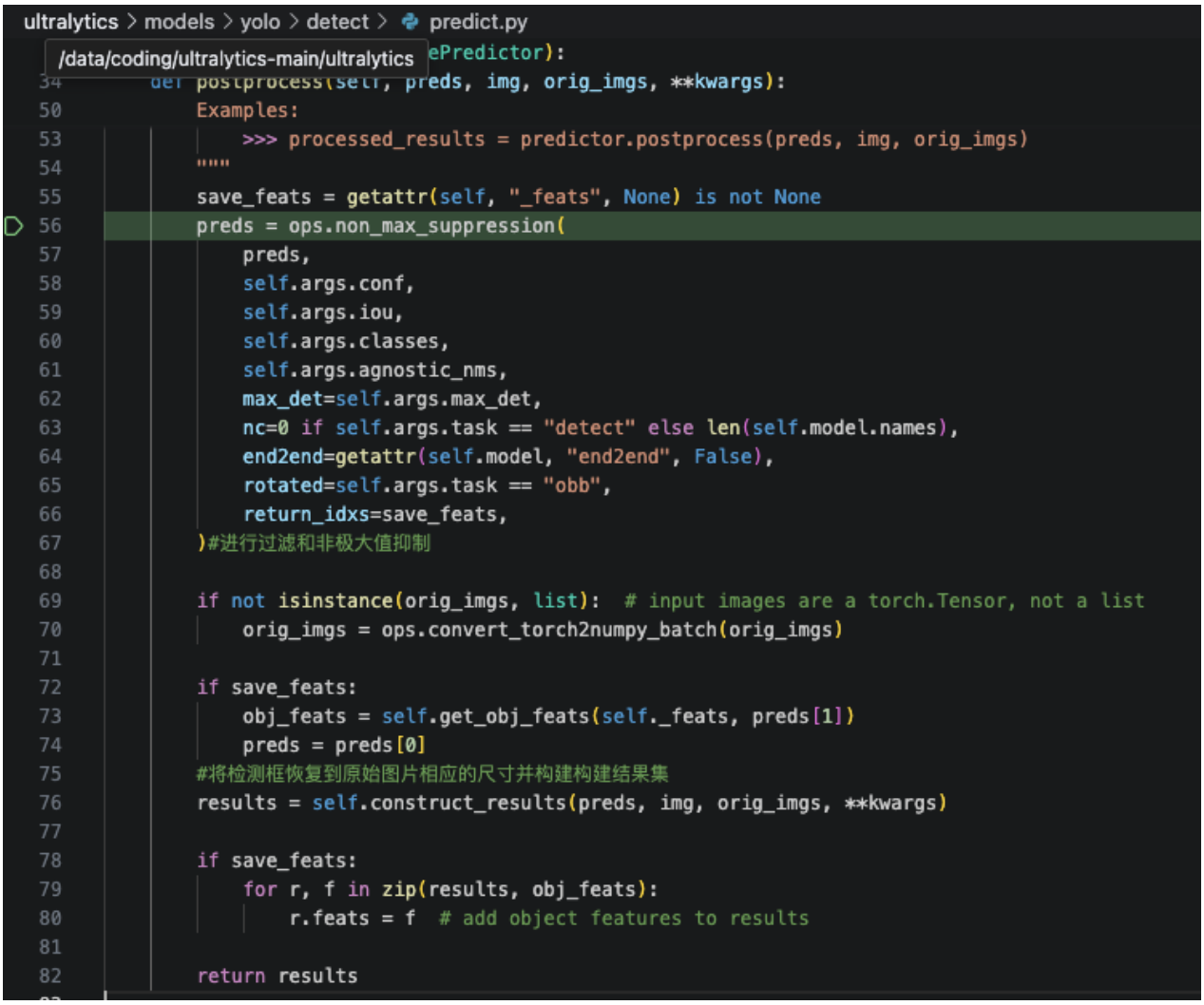
```python
class Detect(nn.Module):
    def forward(self, x: List[torch.Tensor]) -> Union[List[torch.Tensor], Tuple]:
        Concatenate and return predicted bounding boxes and class probabilities.

        前向传播：拼接边界框和类别预测。

        Args:
            x (List[torch.Tensor]): 输入特征图列表
        Returns:
            Union[List[torch.Tensor], Tuple]: 训练时返回特征，推理时返回预测结果
        """
        if self.end2end:
            return self.forward_end2end(x)

        for i in range(self.nl):
            x[i] = torch.cat((self.cv2[i](x[i]), self.cv3[i](x[i])), 1)
        if self.training:  # Training path
            return x
        y = self._inference(x)
        return y if self.export else (y, x)
```

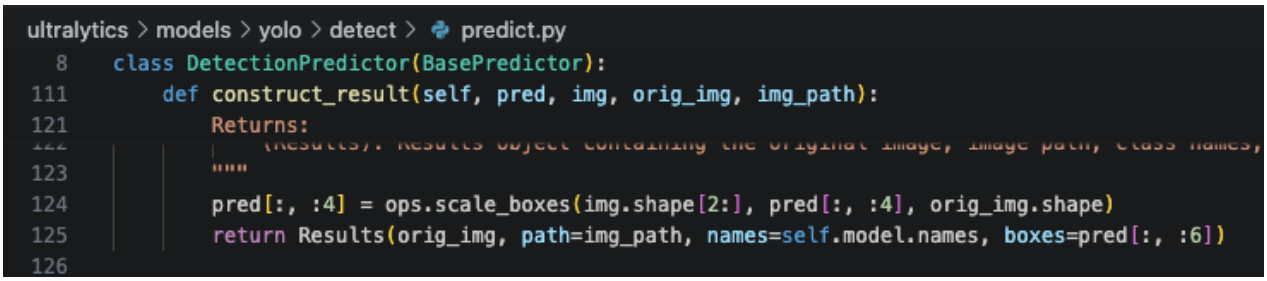
ultralytics > nn > modules > head.py

```python
class Detect(nn.Module):
    def _inference(self, x: List[torch.Tensor]) -> torch.Tensor:
        # 将每层特征图 reshape 为 (B, no, H*W) 并在维度2上拼接
        x_cat = torch.cat([xi.view(shape[0], self.no, -1) for xi in x], 2)

        # 如果不是 IMX 格式且需要重建网格（动态模式或输入形状变化）
        if self.format != "imx" and (self.dynamic or self.shape != shape):
            # 生成锚点和步长
            self.anchors, self.strides = (x.transpose(0, 1) for x in make_anchors(x, self.stride, 0.5))
            self.shape = shape  # 更新当前形状

        # 根据导出格式决定是否使用 split 操作以避免 TF FlexSplitV ops 的问题
        if self.export and self.format in {"saved_model", "pb", "tflite", "edgetpu", "tfjs"}:
            # 分离边界框回归参数和类别预测分数
            box = x_cat[:, : self.reg_max * 4]  # 边界框参数 (B, reg_max*4, N)
            cls = x_cat[:, self.reg_max * 4 :]  # 类别分数 (B, nc, N)
        else:
            # 使用 split 分离边界框和类别
            box, cls = x_cat.split((self.reg_max * 4, self.nc), 1)

        # 针对特定导出格式进行数值稳定性优化
        if self.export and self.format in {"tflite", "edgetpu"}:
            # 预计算归一化因子以提高数值稳定性
            # 参见 https://github.com/ultralytics/ultralytics/issues/7371
            grid_h = shape[2]  # 特征图高度
            grid_w = shape[3]  # 特征图宽度
            # 构造网格尺寸张量 [grid_w, grid_h, grid_w, grid_h]
            grid_size = torch.tensor([grid_w, grid_h, grid_w, grid_h], device=box.device).reshape(1, 4, 1)
            # 计算归一化因子
            norm = self.strides / (self.stride[0] * grid_size)
            # 解码边界框并应用归一化
            dbox = self.decode_bboxes(self.dfl(box) * norm, self.anchors.unsqueeze(0) * norm[:, :2])
        else:
            # 解码边界框（应用 DFL 并乘以步长）
            dbox = self.decode_bboxes(self.dfl(box), self.anchors.unsqueeze(0)) * self.strides
```

1.4 YOLOv11后处理

通过调试分析可知 YOLOv11 的后处理过程在 **ultralytics/models/yolo/detect/predict.py** 文件中：





它包含以下步骤：

- **ops.non_max_suppression**：非极大值抑制，即 NMS
- **ops.scale_boxes**：框的复原，即将最终的检测框恢复到图像原始尺寸的尺度，就是在预处理过程中**resize的逆向操作**。

其中，non_max_suppression的实现如下：

ultralytics > utils > ops.py

```
209     ):
256         bs = prediction.shape[0]  # batch size (BCN, i.e. 1,84,6300)
257         nc = nc or (prediction.shape[1] - 4)  # number of classes
258         extra = prediction.shape[1] - nc - 4  # number of extra info
259         mi = 4 + nc  # mask start index
260         xc = prediction[:, 4:mi].amax(1) > conf_thres  # candidates  过滤类别置信度较低的框
261         xinds = torch.stack([torch.arange(len(i), device=prediction.device) for i in xc])[..., None]  # to track idxs
262 
263         # Settings
264         # min_wh = 2  # (pixels) minimum box width and height
265         time_limit = 2.0 + max_time_img * bs  # seconds to quit after
266         multi_label &= nc > 1  # multiple labels per box (adds 0.5ms/img)
267 
268         prediction = prediction.transpose(-1, -2)  # shape(1,84,6300) to shape(1,6300,84)
269         if not rotated:
270             if in_place:
271                 prediction[..., :4] = xywh2xyxy(prediction[..., :4])  # xywh to xyxy
272             else:
273                 prediction = torch.cat((xywh2xyxy(prediction[..., :4]), prediction[..., 4:]), dim=-1)  # xywh to xyxy
274 
275         t = time.time()
276         output = [torch.zeros((0, 6 + extra), device=prediction.device)] * bs
277         keepi = [torch.zeros((0, 1), device=prediction.device)] * bs  # to store the kept idxs
278         for xi, (x, xk) in enumerate(zip(prediction, xinds)):  # image index, (preds, preds indices)
279             # Apply constraints
280             # x[((x[:, 2:4] < min_wh) | (x[:, 2:4] > max_wh)).any(1), 4] = 0  # width-height
281             filt = xc[xi]  # confidence
282             x, xk = x[filt], xk[filt]
283 
284             # Cat apriori labels if autolabelling
285             if labels and len(labels[xi]) and not rotated:
286                 lb = labels[xi]
287                 v = torch.zeros((len(lb), nc + extra + 4), device=x.device)
288                 v[:, :4] = xywh2xyxy(lb[:, 1:5])  # box
289                 v[range(len(lb)), lb[:, 0].long() + 4] = 1.0  # cls
290                 x = torch.cat((x, v), 0)
```

ultralytics > utils > ops.py

```
209     ):
278         for xi, (x, xk) in enumerate(zip(prediction, xinds)):  # image index, (preds, preds indices)
303             else:  # best class only
308 
309             # Filter by class
310             if classes is not None:
311                 filt = (x[:, 5:6] == classes).any(1)
312                 x, xk = x[filt], xk[filt]
313 
314             # Check shape
315             n = x.shape[0]  # number of boxes
316             if not n:  # no boxes
317                 continue
318             if n > max_nms:  # excess boxes 类别概率排序
319                 filt = x[:, 4].argsort(descending=True)[:max_nms]  # sort by confidence and remove excess boxes
320                 x, xk = x[filt], xk[filt]
321 
322             # Batched NMS
323             c = x[:, 5:6] * (0 if agnostic else max_wh)  # classes
324             scores = x[:, 4]  # scores
325             if rotated:
326                 boxes = torch.cat((x[:, :2] + c, x[:, 2:4], x[:, -1:]), dim=-1)  # xywhr
327                 i = nms_rotated(boxes, scores, iou_thres)
328             else:
329                 boxes = x[:, :4] + c  # boxes (offset by class)
330                 i = torchvision.ops.nms(boxes, scores, iou_thres)  # NMS 操作
331             i = i[:max_det]  # limit detections
332 
333             output[xi], keepi[xi] = x[i], xk[i].reshape(-1)
334             if (time.time() - t) > time_limit:
335                 LOGGER.warning(f"NMS time limit {time_limit:.3f}s exceeded")
336                 break  # time limit exceeded
337 
338         return (output, keepi) if return_idxs else output
```

主要流程包括：

- conf_thres：类别置信度过滤，将类别概率最大值低于conf_thres的检测框过滤掉
- argsort：类别概率排序，为nms做准备
- torchvision.ops.nms：实现nms处理

pytoch实现的后处理代码过于复杂臃肿，梳理后后处理核心代码如下：

```python
def iou(box1, box2):
    def area_box(box):
        return (box[2] - box[0]) * (box[3] - box[1])

    left   = max(box1[0], box2[0])
    top    = max(box1[1], box2[1])
    right  = min(box1[2], box2[2])
    bottom = min(box1[3], box2[3])
    cross  = max((right-left), 0) * max((bottom-top), 0)
    union  = area_box(box1) + area_box(box2) - cross
    if cross == 0 or union == 0:
        return 0
    return cross / union

def NMS(boxes, iou_thres):

    remove_flags = [False] * len(boxes)

    keep_boxes = []
    for i, ibox in enumerate(boxes):
        if remove_flags[i]:
            continue

        keep_boxes.append(ibox)
        for j in range(i + 1, len(boxes)):
            if remove_flags[j]:
                continue

            jbox = boxes[j]
            if(ibox[5] != jbox[5]):
                continue
            if iou(ibox, jbox) > iou_thres:
                remove_flags[j] = True
    return keep_boxes

def postprocess(pred, IM=[], conf_thres=0.25, iou_thres=0.45):

    # 输入是模型推理的结果，即8400个预测框
    # 1,8400,84 [cx,cy,w,h,class*80]
    boxes = []
    for item in pred[0]:
        cx, cy, w, h = item[:4]
        label = item[4:].argmax()
```

```
        confidence = item[4 + label]
        if confidence < conf_thres:
            continue
        left    = cx - w * 0.5
        top     = cy - h * 0.5
        right   = cx + w * 0.5
        bottom  = cy + h * 0.5
        boxes.append([left, top, right, bottom, confidence, label])

    boxes = np.array(boxes)
    lr = boxes[:,[0, 2]]
    tb = boxes[:,[1, 3]]
    boxes[:,[0,2]] = IM[0][0] * lr + IM[0][2]
    boxes[:,[1,3]] = IM[1][1] * tb + IM[1][2]
    boxes = sorted(boxes.tolist(), key=lambda x:x[4], reverse=True)

    return NMS(boxes, iou_thres)

```

其中预测框的复原是通过仿射变换逆矩阵 IM 实现的，这个IM矩阵是在前处理resize的时候计算出来的，因此在后续的步数过程中，前处理步骤需要将IM矩阵传递给后处理步骤，因为这个矩阵针对不同尺寸的图像是不一样的。

1.5 YOLOv11推理python实现

通过上面对 YOLO11 的预处理、模型推理和后处理分析之后，整个推理过程就显而易见了。YOLO11 的推理包括图像预处理、模型推理、预测结果后处理三部分，其中预处理主要包括 **warpAffine 仿射变换**，后处理主要包括 **预测框复原（也是warpAffine 仿射变换）和 NMS** 两部分。

在项目主目录中新建文件`infer.py`，输入如下代码：

```
import cv2
import torch
import numpy as np
from ultralytics.data.augment import LetterBox
from ultralytics.nn.autobackend import AutoBackend

def preprocess_letterbox(image):
    letterbox = LetterBox(new_shape=640, stride=32, auto=True)
    image = letterbox(image=image)
    image = (image[..., ::-1] / 255.0).astype(np.float32) # BGR to RGB, 0
- 255 to 0.0 - 1.0
    image = image.transpose(2, 0, 1)[None]  # BHWC to BCHW (n, 3, h, w)
    image = torch.from_numpy(image)
    return image

def preprocess_warpAffine(image, dst_width=640, dst_height=640):
    scale = min((dst_width / image.shape[1], dst_height /
image.shape[0]))
```

```python
    ox = (dst_width  - scale * image.shape[1]) / 2
    oy = (dst_height - scale * image.shape[0]) / 2
    M = np.array([
        [scale, 0, ox],
        [0, scale, oy]
    ], dtype=np.float32)

    img_pre = cv2.warpAffine(image, M, (dst_width, dst_height), flags=cv2.INTER_LINEAR,
                             borderMode=cv2.BORDER_CONSTANT, borderValue=(114, 114, 114))
    IM = cv2.invertAffineTransform(M)

    img_pre = (img_pre[...,::-1] / 255.0).astype(np.float32)
    img_pre = img_pre.transpose(2, 0, 1)[None]
    img_pre = torch.from_numpy(img_pre)
    return img_pre, IM

def iou(box1, box2):
    def area_box(box):
        return (box[2] - box[0]) * (box[3] - box[1])

    left   = max(box1[0], box2[0])
    top    = max(box1[1], box2[1])
    right  = min(box1[2], box2[2])
    bottom = min(box1[3], box2[3])
    cross  = max((right-left), 0) * max((bottom-top), 0)
    union  = area_box(box1) + area_box(box2) - cross
    if cross == 0 or union == 0:
        return 0
    return cross / union

def NMS(boxes, iou_thres):

    remove_flags = [False] * len(boxes)

    keep_boxes = []
    for i, ibox in enumerate(boxes):
        if remove_flags[i]:
            continue

        keep_boxes.append(ibox)
        for j in range(i + 1, len(boxes)):
            if remove_flags[j]:
                continue

            jbox = boxes[j]
            if(ibox[5] != jbox[5]):
                continue
```

```
            if iou(ibox, jbox) > iou_thres:
                remove_flags[j] = True
    return keep_boxes

def postprocess(pred, IM=[], conf_thres=0.25, iou_thres=0.45):

    # 输入是模型推理的结果，即8400个预测框
    # 1,8400,84 [cx,cy,w,h,class*80]
    boxes = []
    for item in pred[0]:
        cx, cy, w, h = item[:4]
        label = item[4:].argmax()
        confidence = item[4 + label]
        if confidence < conf_thres:
            continue
        left    = cx - w * 0.5
        top     = cy - h * 0.5
        right   = cx + w * 0.5
        bottom  = cy + h * 0.5
        boxes.append([left, top, right, bottom, confidence, label])

    boxes = np.array(boxes)
    lr = boxes[:,[0, 2]]
    tb = boxes[:,[1, 3]]
    boxes[:,[0,2]] = IM[0][0] * lr + IM[0][2]
    boxes[:,[1,3]] = IM[1][1] * tb + IM[1][2]
    boxes = sorted(boxes.tolist(), key=lambda x:x[4], reverse=True)

    return NMS(boxes, iou_thres)

def hsv2bgr(h, s, v):
    h_i = int(h * 6)
    f = h * 6 - h_i
    p = v * (1 - s)
    q = v * (1 - f * s)
    t = v * (1 - (1 - f) * s)

    r, g, b = 0, 0, 0

    if h_i == 0:
        r, g, b = v, t, p
    elif h_i == 1:
        r, g, b = q, v, p
    elif h_i == 2:
        r, g, b = p, v, t
    elif h_i == 3:
        r, g, b = p, q, v
    elif h_i == 4:
        r, g, b = t, p, v
```

```
    elif h_i == 5:
        r, g, b = v, p, q

    return int(b * 255), int(g * 255), int(r * 255)

def random_color(id):
    h_plane = (((id << 2) ^ 0x937151) % 100) / 100.0
    s_plane = (((id << 3) ^ 0x315793) % 100) / 100.0
    return hsv2bgr(h_plane, s_plane, 1)

if __name__ == "__main__":

    img = cv2.imread("/data/coding/yolo_data/val/images/00002.jpg")

    # img_pre = preprocess_letterbox(img)
    img_pre, IM = preprocess_warpAffine(img)

    model  = AutoBackend(weights="/data/coding/ultralytics-main/runs/detect/train/weights/best.pt")
    names  = model.names
    result = model(img_pre)[0].transpose(-1, -2)  # 1,6300,73

    boxes  = postprocess(result, IM)

    for obj in boxes:
        left, top, right, bottom = int(obj[0]), int(obj[1]), int(obj[2]), int(obj[3])
        confidence = obj[4]
        label = int(obj[5])
        color = random_color(label)
        cv2.rectangle(img, (left, top), (right, bottom), color=color,thickness=2, lineType=cv2.LINE_AA)
        caption = f"{names[label]} {confidence:.2f}"
        w, h = cv2.getTextSize(caption, 0, 1, 2)[0]
        cv2.rectangle(img, (left - 3, top - 33), (left + w + 10, top), color, -1)
        cv2.putText(img, caption, (left, top - 5), 0, 1, (0, 0, 0), 2, 16)

    cv2.imwrite("infer.jpg", img)
    print("save done")

```

执行后查看结果如下，可以看到，跟之前的版本结果一致。

2 CUDA基础

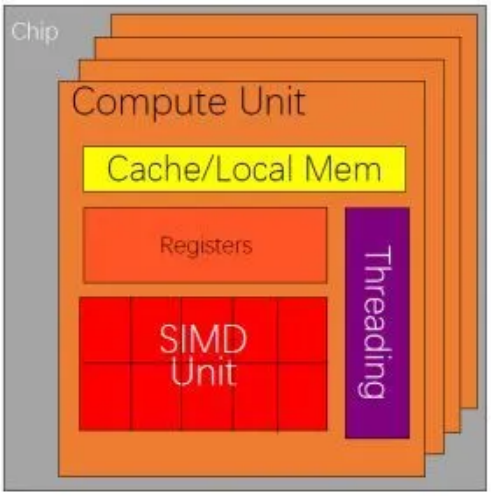
2.1 CPU 和 GPU 的基础知识

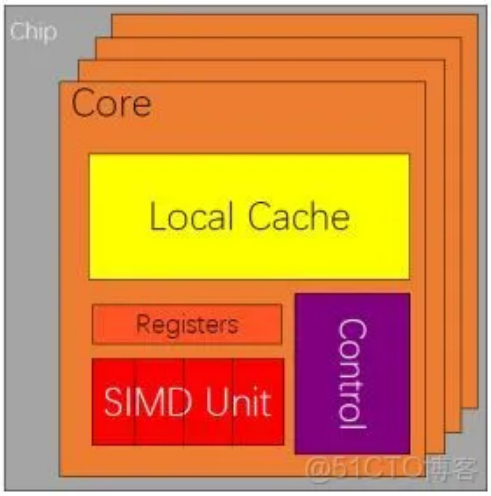
提到处理器结构，有2个指标是经常要考虑的：**延迟**和**吞吐量**。

- 延迟：从发出指令到最终返回结果中间经历的时间间隔。
- 吞吐量：单位时间内处理的指令的条数。

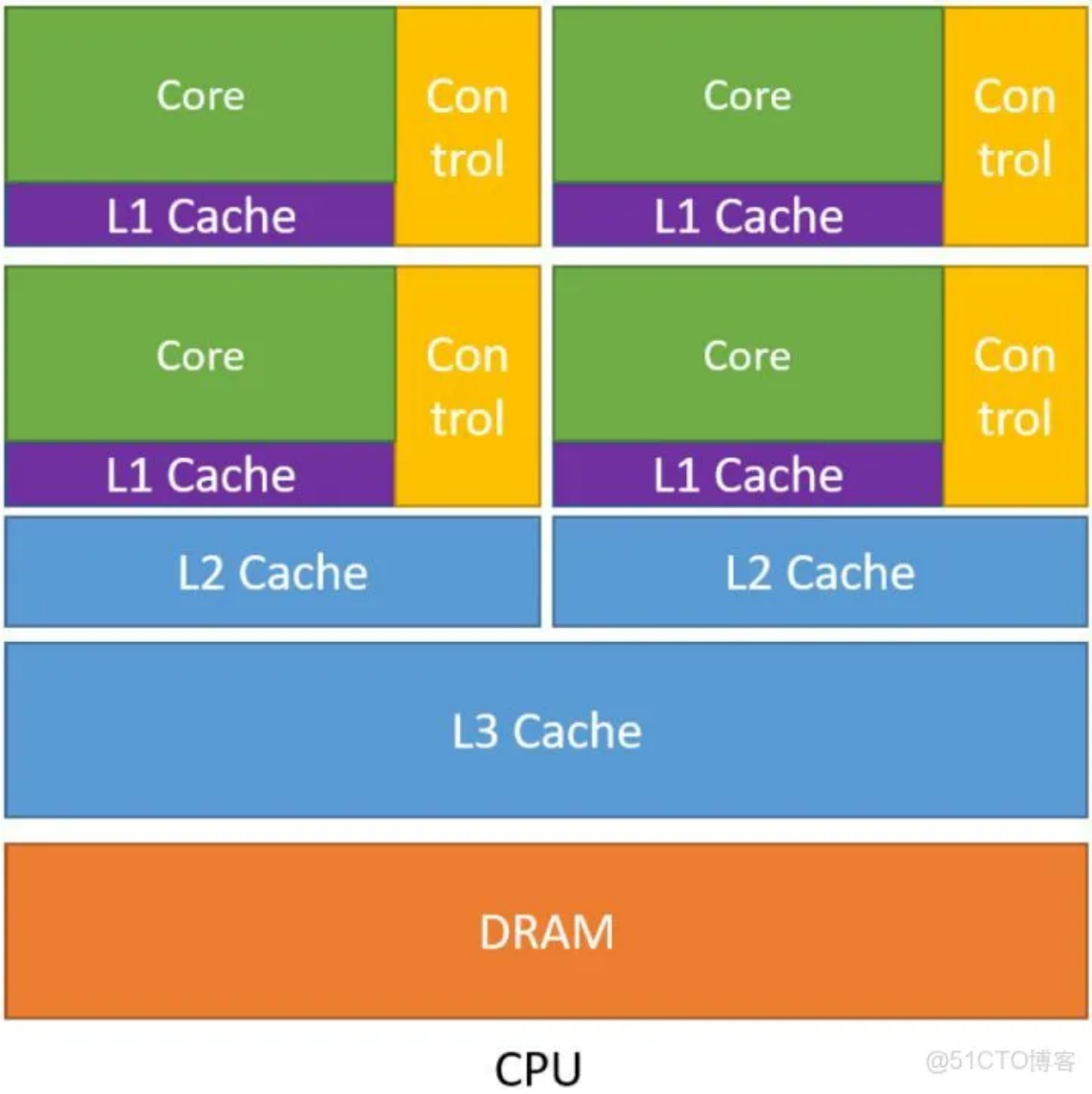
GPU
吞吐导向内核

CPU
延迟导向内核





2.1.1 CPU架构

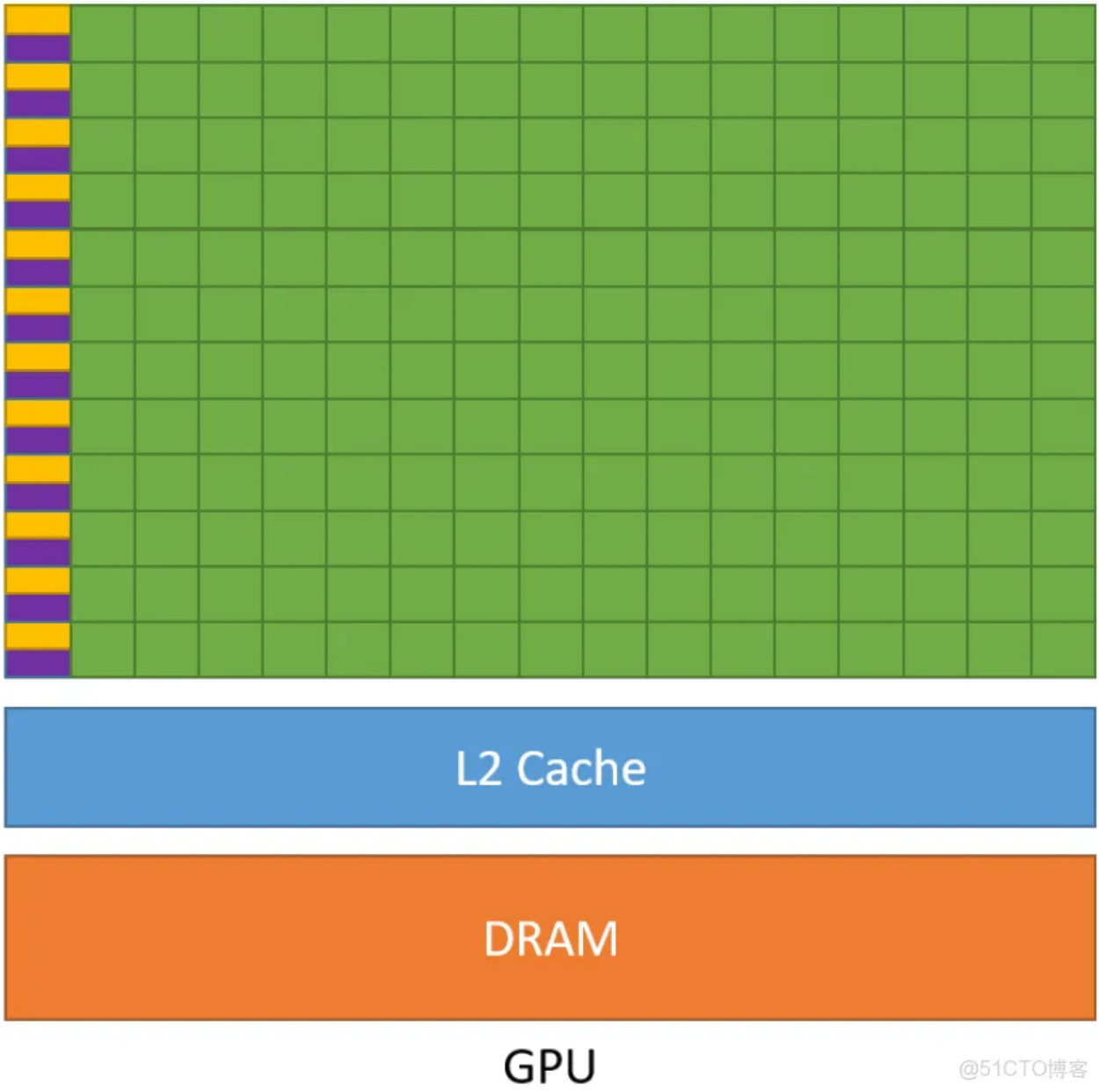


从图中可以看出 CPU 的几个特点：

- CPU 中包含了多级高速的缓存结构。 运算的速度远高于访问存储的速度，那么奔着空间换时间的思想，设计了多级高速的缓存结构，将经常访问的内容放到低级缓存中，将不经常访问的内容放到高级缓存中，从而提升了指令访问存储的速度。
- CPU 中包含了很多控制单元。 具体有2种，一个是**分支预测**机制，另一个是流水线前传机制。
- CPU 的运算单元 (Core) 强大，整型浮点型复杂运算**速度快**，但是核心**数量少**。

CPU 在设计时的导向就是减少指令的时延，我们称之为**延迟导向**设计。

2.1.2 GPU架构



从图中可以看出 GPU 的几个特点 (注意紫色和黄色的区域分别是缓存单元和控制单元)：

- GPU 中虽有缓存结构但是数量少。 因为要减少指令访问缓存的次数。
- GPU 中控制单元非常简单。 控制单元中也没有分支预测机制和数据转发机制。对于复杂的指令运算就会比较慢。
- GPU 的运算单元 (Core) **非常多**，采用长延时流水线以实现高吞吐量。 每一行的运算单元的控制器只有一个，意味着每一行的运算单元使用的指令是相同的，不同的是它们的数据内容。那么这种整齐划一的运算方式使得 GPU 对于那些控制简单但运算高效的指令的效率显著增加。

GPU 在设计过程中以一个原则为核心：增加简单指令的吞吐。因此，我们称 GPU 为**吞吐导向**设计。

那么究竟在什么情况下使用 CPU，什么情况下使用 GPU 呢?

- CPU 在连续计算部分，延迟优先，CPU 比 GPU ，单条复杂指令延迟快10倍以上。
- GPU 在并行计算部分，吞吐优先，GPU 比 CPU ，单位时间内执行指令数量10倍以上。

适合 GPU 的问题：

- **计算密集**：数值计算的比例要远大于内存操作，因此内存访问的延时可以被计算掩盖。
- **数据并行**：大任务可以拆解为执行相同指令的小任务，因此对复杂流程控制的需求较低。

GPU实际架构图（Ampere）

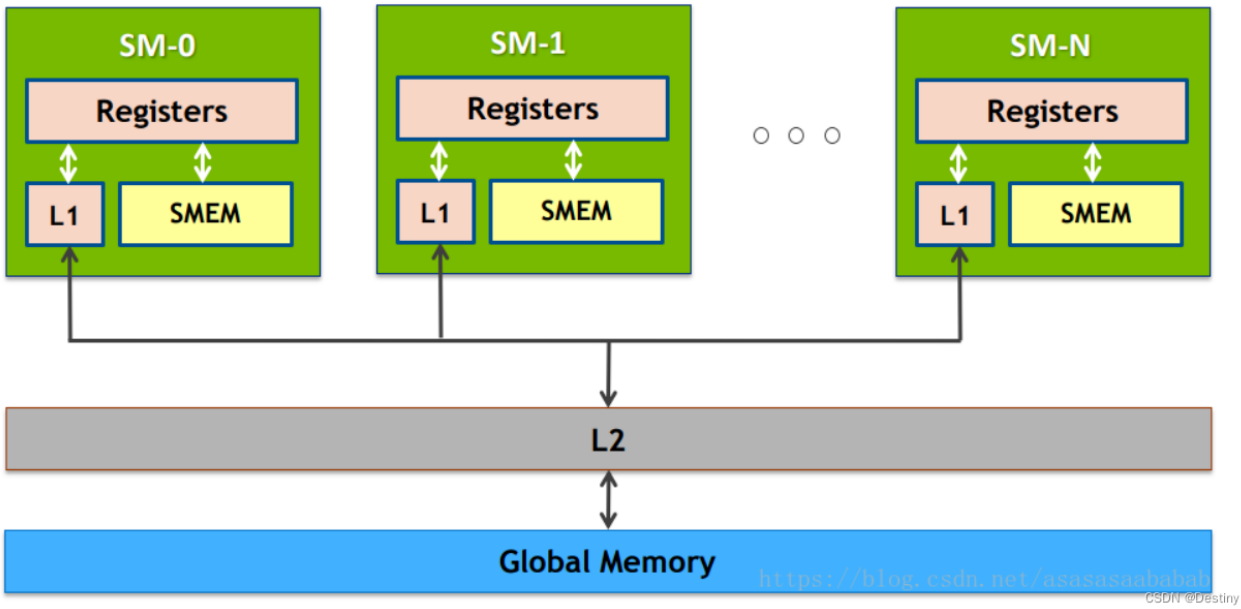
GA100 的 SM 结构图

- CUDA Core: 负责执行单精度操作，比如FMUL、FADD和FFMA等。为了提高指令的throughput，CUDA Core是pipelined。
- tensorcore :执行矩阵乘法MMA指令。
- INT32、FP32、FP64: 负责执行整数、单精度(半精度)、双精度（duoble precision）指令。
- LD/ST：负责memory load，store等操作。
- SFU：特殊函数单元（Special Function Unit）负责计算特殊的函数，比如cuda中的**cosf()**、

expf() intrinsics就调用了SFU单元。需要注意的是，SFU实现的是快速近似操作，可能会影响精度。

- register file：寄存器组，用来存储数据
- dispatch unit：负责将不同类型的指令要被分配到不同的单元上去执行
- warp scheduler：负责管理一系列的warp，并从中选取符合条件的warp发射指令

GPU的内存架构



2.2 CUDA 编程的重要概念

CUDA (Compute Unified Device Architecture)，由英伟达公司2007年开始推出，初衷是为 GPU 增加一个易用的编程接口，让开发者无需学习复杂的着色语言或者图形处理原语。

OpenCL (Open Computing Languge) 是2008年发布的异构平台并行编程的开放标准，也是一个编程框架。OpenCL 相比 CUDA，支持的平台更多，除了 GPU 还支持 CPU、DSP、FPGA 等设备。

2.2.1 CUDA编程模型

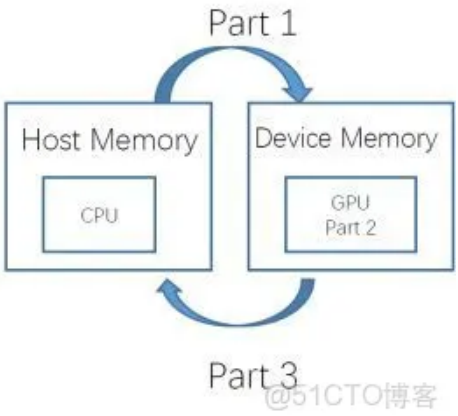
CUDA的架构中引入了主机端（host）和设备（device）的概念。CUDA程序中既包含host程序，又包含device程序。同时，host与device之间可以进行通信，这样它们之间可以进行数据拷贝。

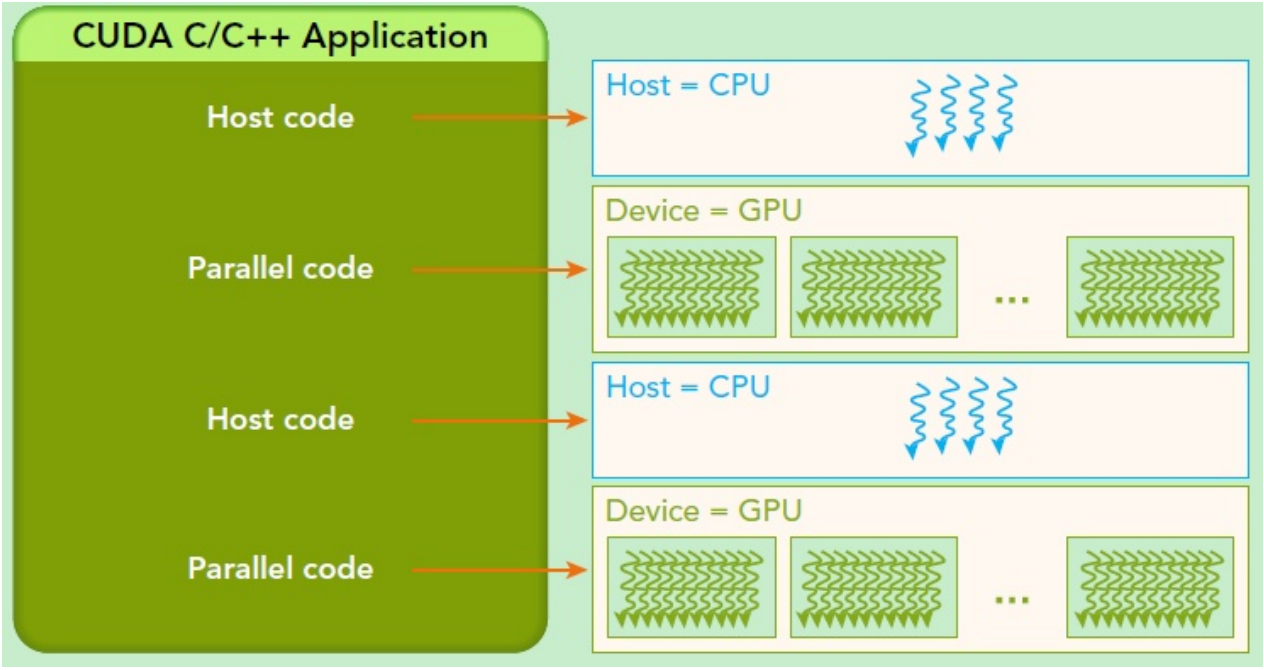
- **主机(Host)**：将CPU及系统的内存（内存条）称为主机。
- **设备(Device)**：将GPU及GPU本身的显示内存称为设备。

典型的CUDA程序的执行流程如下：

1. 分配host内存，并进行数据初始化；
2. 分配device内存，并从host将数据拷贝到device上；
3. 调用CUDA的核函数在device上完成指定的运算；
4. 将device上的运算结果拷贝到host上；
5. 释放device和host上分配的内存。

```
void GPUkernel(float* A, float* B, float* C, int n)
{
1. // Allocate device memory for A, B, and C
   // copy A and B to device memory

2. // Kernel launch code - to have the device
   // to perform the actual vector addition

3. // copy C from the device memory
   // Free device vectors
}
```





2.2.2 线程层次结构

2.2.2.1 软件层次

核 kernel

CUDA执行流程中最重要的一个过程是调用CUDA的核函数来执行并行计算，kernel是CUDA中一个重要的概念。在CUDA程序构架中，主机端代码部分在CPU上执行，是普通的C代码；当遇到数据并行处理的部分，CUDA 就会将程序编译成GPU能执行的程序，并传送到GPU，这个程序在CUDA里称做核（kernel）。设备端代码部分在GPU上执行，此代码部分在kernel上编写（.cu文件）。kernel用**global**符号声明，在调用时需要用<<<grid, block>>>来指定kernel要执行及结构。

CUDA是通过函数类型限定词区别在host和device上的函数，主要的三个函数类型限定词如下：

- **global**：在device上执行，从host中调用（一些特定的GPU也可以从device上调用），返回类型必须是void，不支持可变参数参数，不能成为类成员函数。注意用**global**定义的kernel是异步的，这意味着host不会等待kernel执行完就执行下一步。
- **device**：在device上执行，单仅可以从device中调用，不可以和**global**同时用。
- **host**：在host上执行，仅可以从host上调用，一般省略不写，不可以和**global**同时用，但可和**device**同时使用，此时函数会在device和host都编译。

网格 grid：kernel在device上执行时，实际上是启动很多线程，一个kernel所启动的所有线程称为一个网格（grid），同一个网格上的线程共享相同的全局内存空间。grid是线程结构的第一层次。

线程块 block：网格又可以分为很多线程块（block），一个block里面包含很多线程。各block是并行执行的，block间无法通信，也没有执行顺序。block的数量限制为不超过65535（硬件限制）。第二层次。

grid和block都是定义为dim3类型的变量，dim3可以看成是包含三个无符号整数（x，y，z）成员的结构体变量，在定义时，缺省值初始化为1。grid和block可以灵活地定义为1-dim，2-dim以及3-dim结构。

CUDA中，每一个线程都要执行核函数，每一个线程需要kernel的两个内置坐标变量（blockIdx，threadIdx）来唯一标识，其中blockIdx指明线程所在grid中的位置，threaIdx指明线程所在block中的位置。它们都是dim3类型变量。

一个线程在block中的全局ID，必须还要知道block的组织结构，这是通过线程的内置变量blockDim来获得。它获取block各个维度的大小。对于一个2-dim的block（D_x,D_y），线程 (x,y) 的ID值为 (x+y∗D_x)，如果是3-dim的block (D_x,D_y,D_z)，线程(x,y,z)的ID值为(x+y∗D_x+z∗D_x∗D_y) 。另外线程还有内置变量gridDim，用于获得grid各个维度的大小。

每个block有包含共享内存（Shared Memory）,可以被线程块中所有线程共享，其生命周期与线程块一致。 每个thread有自己的私有本地内存（Local Memory）。此外，所有的线程都可以访问全局内存（Global Memory），还可以访问一些只读内存块：常量内存（Constant Memory）和纹理内存（Texture Memory）。

线程 thread：一个CUDA的并行程序会被以许多个threads来执行。数个threads会被群组成一个block，同一个block中的threads可以同步，也可以通过shared memory通信。

线程束 warp：GPU执行程序时的调度单位，SM的基本执行单元。目前在CUDA架构中，warp是一个包含32个线程的集合，这个线程集合被“编织在一起”并且“步调一致”的形式执行。同一个warp中的每个线程都将以不同数据资源执行相同的指令，这就是所谓 SIMT架构(Single-Instruction, Multiple-Thread，单指令多线程)。

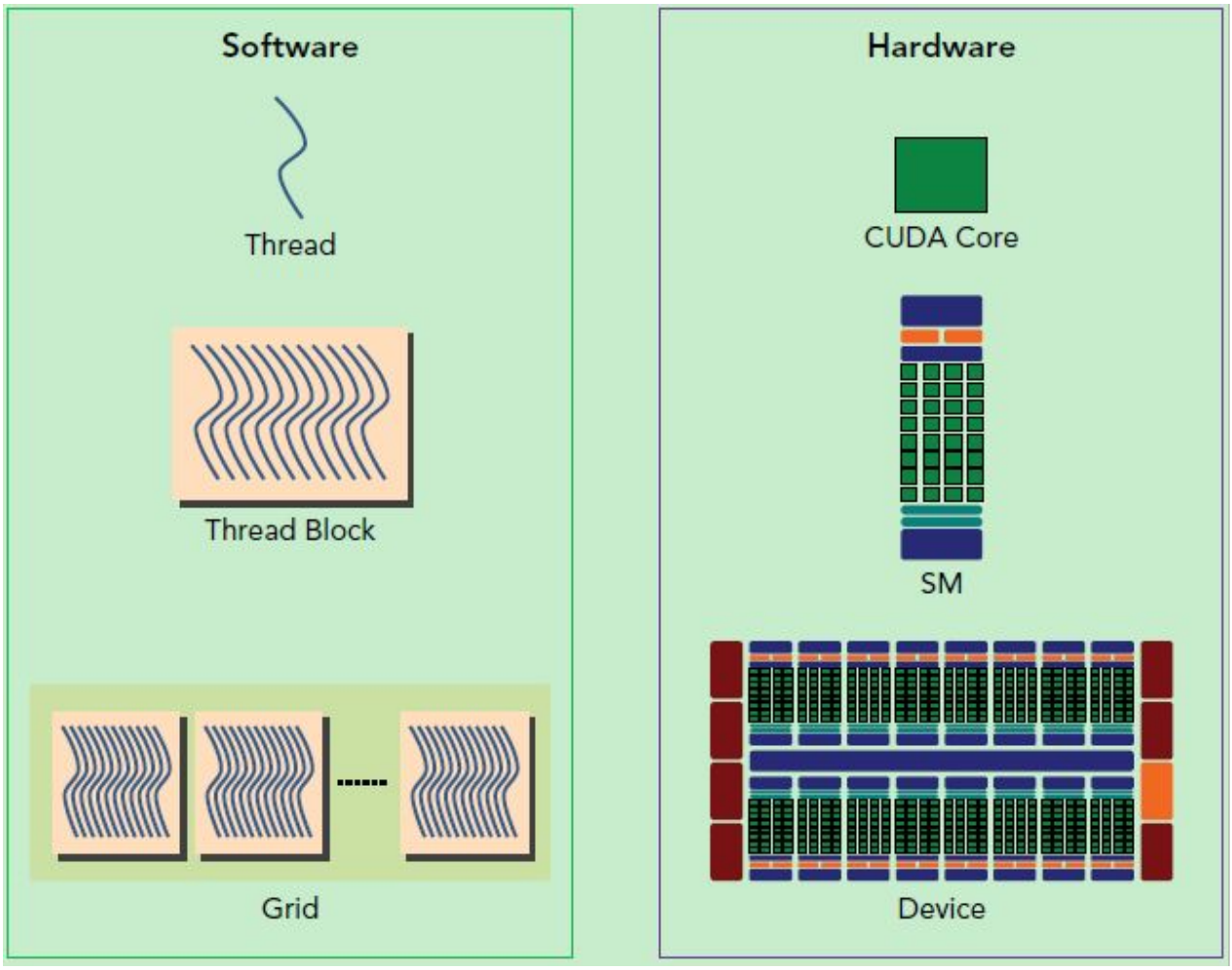
2.2.2.2 硬件层次

SP：最基本的处理单元，streaming processor，也称为CUDA core。最后具体的指令和任务都是在SP上处理的。GPU进行并行计算，也就是很多个SP同时做处理。

SM：GPU硬件的一个核心组件是流式多处理器（Streaming Multiprocessor）。SM的核心组件包括CUDA核心、共享内存、寄存器等。SM可以并发地执行数百个线程。一个block上的线程是放在同一个流式多处理器（SM）上的，因而，一个SM的有限存储器资源制约了每个block的线程数量。在早期的NVIDIA 架构中，一个block最多可以包含 512个线程，而在后期出现的一些设备中则最多可支持1024个线程。一个kernel可由多个大小相同的block同时执行，因而线程总数应等于每个块的线程数乘以块的数量。

软硬件的对应关系如下：

- 线程处理器 (SP) 对应线程 (thread)。
- 多核处理器 (SM) 对应线程块 (thread block)。
- 设备端 (device) 对应线程块组合体 (grid)。



一个kernel实际会启动很多线程，这些线程是逻辑上并行的，但是网格和线程块只是逻辑划分，SM才是执行的物理层，在物理层并不一定同时并发。原因如下：

1. 当一个kernel被执行时，它的gird中的block被分配到SM上，一个block只能在一个SM上被调度。SM一般可以调度多个block，这要看SM本身的能力。有可能一个kernel的各个block被分配至多个SM上，所以grid只是逻辑层，SM才是执行的物理层。
2. 当block被划分到某个SM上时，它将进一步划分为多个wraps。SM采用的是SIMT，基本的执行单元是wraps，一个wrap包含32个线程，这些线程同时执行相同的指令，但是每个线程都包含自己的指令地址计数器和寄存器状态，也有自己独立的执行路径。所以尽管wraps中的线程同时从同一程序地址执行，但是可能具有不同的行为，比如遇到了分支结构，一些线程可能进入这个分支，但是另外一些有可能不执行，它们只能死等，因为GPU规定warp中所有线程在同一周期执行相同的指令，线程束分化会导致性能下降。

综上，SM要为每个block分配shared memory，而也要为每个warp中的线程分配独立的寄存器。所以SM的配置会影响其所支持的线程块和线程束并发数量。所以kernel的grid和block的配置不同，性能会出现差异。还有，由于SM的基本执行单元是包含32个线程的warp，所以block大小一般要设置为32的倍数。

2.3 向量加法的cuda实践

代码实现：

```
#include <stdio.h>
#include <stdlib.h>
```

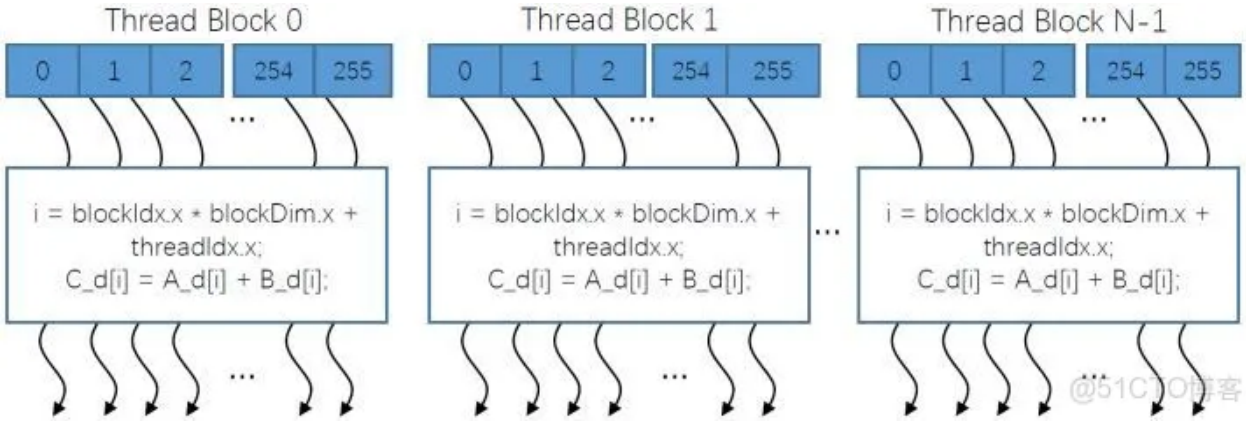
```cpp
#include <math.h>
#include <cuda_runtime.h>

// CUDA核函数：向量加法
__global__ void vectorAdd(const float *A_d, const float *B_d, float *C_d, int numElements) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;//不同的线程读取不同位置的数据进行计算

    if (i < numElements) {
        C_d[i] = A_d[i] + B_d[i];
    }
}

int main(int argc, char *argv[]) {
    // 检查命令行参数
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <numElements>\n", argv[0]);
        return EXIT_FAILURE;
    }

    // 向量元素数量
    int numElements = atoi(argv[1]);
    printf("Number of elements: %d\n", numElements);
    size_t size = numElements * sizeof(float);

    // 分配主机内存
    float *h_A = (float *)malloc(size);
    float *h_B = (float *)malloc(size);
    float *h_C = (float *)malloc(size);

    if (h_A == NULL || h_B == NULL || h_C == NULL) {
        fprintf(stderr, "Failed to allocate host memory!\n");
        return EXIT_FAILURE;
    }

    // 初始化输入向量
    for (int i = 0; i < numElements; ++i) {
        h_A[i] = rand() / (float)RAND_MAX;
        h_B[i] = rand() / (float)RAND_MAX;
    }

    // 分配设备内存
    float *d_A = NULL;
    float *d_B = NULL;
    float *d_C = NULL;
    cudaError_t err = cudaMalloc((void **)&d_A, size);
    if (err != cudaSuccess) {
```

```
        fprintf(stderr, "Failed to allocate device memory for A: %s\n", cudaGetErrorString(err));
        return EXIT_FAILURE;
    }
    err = cudaMalloc((void **)&d_B, size);
    if (err != cudaSuccess) {
        fprintf(stderr, "Failed to allocate device memory for B: %s\n", cudaGetErrorString(err));
        cudaFree(d_A);
        return EXIT_FAILURE;
    }
    err = cudaMalloc((void **)&d_C, size);
    if (err != cudaSuccess) {
        fprintf(stderr, "Failed to allocate device memory for C: %s\n", cudaGetErrorString(err));
        cudaFree(d_A);
        cudaFree(d_B);
        return EXIT_FAILURE;
    }

    // 将数据从主机复制到设备
    err = cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    if (err != cudaSuccess) {
        fprintf(stderr, "Failed to copy A to device: %s\n", cudaGetErrorString(err));
        cudaFree(d_A);
        cudaFree(d_B);
        cudaFree(d_C);
        return EXIT_FAILURE;
    }
    err = cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
    if (err != cudaSuccess) {
        fprintf(stderr, "Failed to copy B to device: %s\n", cudaGetErrorString(err));
        cudaFree(d_A);
        cudaFree(d_B);
        cudaFree(d_C);
        return EXIT_FAILURE;
    }

    // 启动核函数
    int threadsPerBlock = 256;//32
    int blocksPerGrid = (numElements + threadsPerBlock - 1) / threadsPerBlock;//256/32=8

    printf("Launching kernel with %d blocks of %d threads each\n", blocksPerGrid, threadsPerBlock);
    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, numElements);
```

```
90
91      // 检查核函数执行是否有错误
92      err = cudaGetLastError();
93      if (err != cudaSuccess) {
94          fprintf(stderr, "Failed to launch kernel: %s\n",
    cudaGetErrorString(err));
95          cudaFree(d_A);
96          cudaFree(d_B);
97          cudaFree(d_C);
98          return EXIT_FAILURE;
99      }
100
101     // 将结果从设备复制回主机
102     err = cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
103     if (err != cudaSuccess) {
104         fprintf(stderr, "Failed to copy C from device: %s\n",
    cudaGetErrorString(err));
105         cudaFree(d_A);
106         cudaFree(d_B);
107         cudaFree(d_C);
108         return EXIT_FAILURE;
109     }
110
111     // 验证结果
112     for (int i = 0; i < numElements && i < 50; ++i) {
113         if (fabs(h_A[i] + h_B[i] - h_C[i]) > 1e-5) {
114             fprintf(stderr, "Result verification failed at element
    %d!\n", i);
115             cudaFree(d_A);
116             cudaFree(d_B);
117             cudaFree(d_C);
118             free(h_A);
119             free(h_B);
120             free(h_C);
121             return EXIT_FAILURE;
122         }
123     }
124
125     printf("Test PASSED\n");
126
127     // 释放设备内存
128     cudaFree(d_A);
129     cudaFree(d_B);
130     cudaFree(d_C);
131
132     // 释放主机内存
133     free(h_A);
134     free(h_B);
135     free(h_C);
```

```
136
137     return 0;
138 }
```

CUDA 核函数由线程网格 (数组) 执行。每个线程都有一个索引，用于计算内存地址和做出控制决策。在计算完以后 (图中所有弯箭头的头部)，会设置一个时钟，将这N个线程块的计算结果进行同步。



编译程序：

```
1 nvcc -arch=sm_86 vectorAdd.cu -o vectorAdd
```

执行程序：

```
1 ./vectorAdd 1000000000
```

3 YOLOv11推理(C++)

> C++ 上的实现我们使用 tensorRT_Pro这个项目，现在我们就基于 tensorRT_Pro 完成 YOLO11 在 C++ 上的推理。项目地址：tensorRT_Pro-YOLOv8

3.1 ONNX导出

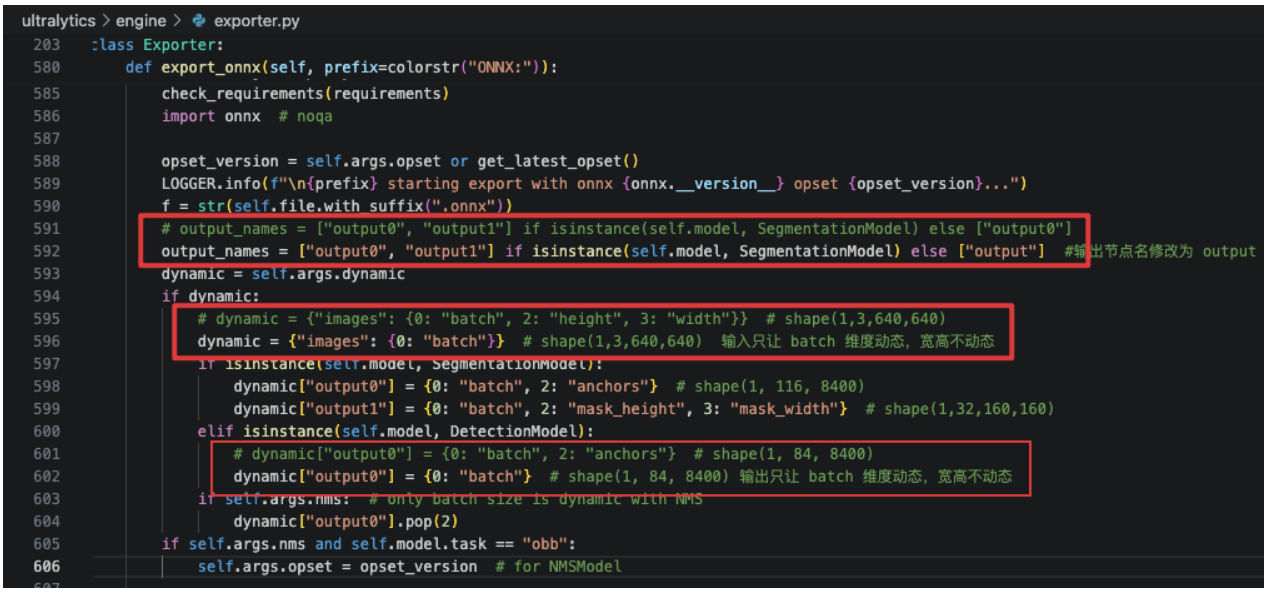
首先我们需要将 YOLO11 模型导出为 ONNX，为了适配 tensorRT_Pro 我们需要做一些修改

- 修改输出节点名为 output，输入输出只让 batch 维度动态，宽高不动态
- 增加 transpose 节点交换输出的 2、3 维度

具体修改如下：

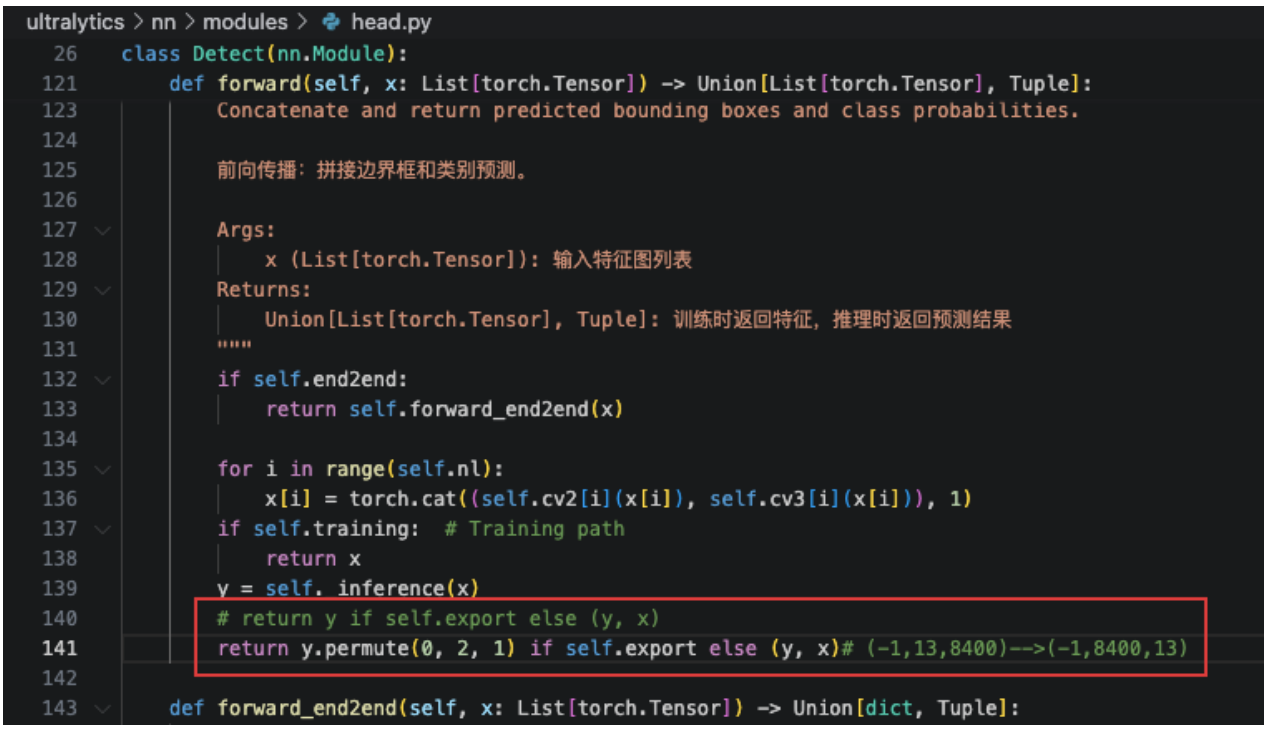
1. 在 ultralytics/engine/exporter.py 文件中改动一处

- 591 行：输出节点名修改为 output
- 594 行：输入只让 batch 维度动态，宽高不动态
- 599 行：输出只让 batch 维度动态，宽高不动态



2. 在 ultralytics/nn/modules/head.py 文件中改动一处

- **140行**：添加 **transpose** 节点交换输出的第 2 和第 3 维度



在 ultralytics-main 主目录下，新建导出文件 **export.py**，内容如下：

```python
from ultralytics import YOLO

model = YOLO("/data/coding/ultralytics-main/runs/detect/train/weights/best.pt")

success = model.export(format="onnx", dynamic=True, simplify=True)
```

在终端执行如下指令即可完成 onnx 导出：

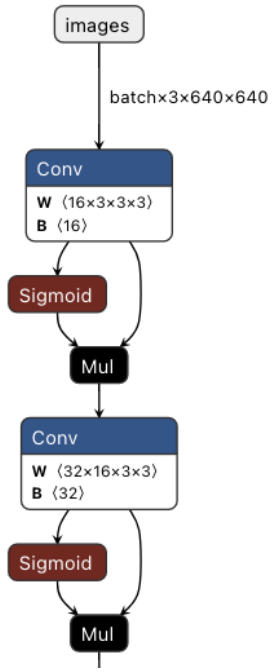
```
python export.py
```

导出过程如下图所示：

```
(torch) root@jrvafckrepigddhj-snow-6bb675885f-mcm6g:/data/coding/ultralytics-main# python export.py
Ultralytics 8.3.168 🚀 Python-3.10.16 torch-2.7.1+cu126 CPU (Intel Xeon E5-2683 v4 2.10GHz)
YOLO11n summary (fused): 100 layers, 2,583,907 parameters, 0 gradients, 6.3 GFLOPs

PyTorch: starting from '/data/coding/ultralytics-main/runs/detect/train/weights/best.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) (1, 8400, 1
3) (5.2 MB)
requirements: Ultralytics requirements ['onnx>=1.12.0,<1.18.0', 'onnxslim>=0.1.59', 'onnxruntime'] not found, attempting AutoUpdate...
^CWARNING ⚠️

ONNX: starting export with onnx 1.19.0 opset 19...
ERROR: Operation cancelled by user
WARNING ⚠️ONNX: simplifier failure: No module named 'onnxslim'
ONNX: export success ✅ 8.7s, saved as '/data/coding/ultralytics-main/runs/detect/train/weights/best.onnx' (9.9 MB)

Export complete (9.2s)
Results saved to /data/coding/ultralytics-main/runs/detect/train/weights
Predict:         yolo predict task=detect model=/data/coding/ultralytics-main/runs/detect/train/weights/best.onnx imgsz=640
Validate:        yolo val task=detect model=/data/coding/ultralytics-main/runs/detect/train/weights/best.onnx imgsz=640 data=data.yaml
Visualize:       https://netron.app
```

可以看到导出的 pytorch 模型的输入 shape 是 1x3x640x640，输出 shape 是 1x8400x13，符合我们的预期。

导出成功后会在/data/coding/ultralytics-main/runs/detect/train/weights目录下生成best.onnx 模型，我们可以使用 Netron 可视化工具查看，如下图所示：



可以看到输入节点名是 **images**，维度是 batchx3x640x640，保证只有 batch 维度动态，输出节点名是 **output**，维度是 batchx8400x13，保证只有 batch 维度动态，符合 tensorRT_Pro 的格式。

3.2 环境配置

3.2.1 安装依赖库

需要使用的软件环境有 **TensorRT**、**OpenCV**、**cuDNN**、**Protobuf**。

3.2.1.1 TensorRT安装

TensorRT的版本是根据CUDA版本和cuDNN版本来的，我的环境是CUDA 12.6.85 版本，依据此来安装TensorRT。

> 如何查看cuda版本?

```
nvcc -V
```

```
(torch) root@cdjwpfdtbspbbigt-snow-57957dc8d7-rmnd8:/data/coding# nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Tue_Oct_29_23:50:19_PDT_2024
Cuda compilation tools, release 12.6, V12.6.85
Build cuda_12.6.r12.6/compiler.35059454_0
```

查看系统版本：

```
cat /etc/os-release
```

```
(torch) root@cdjwpfdtbspbbigt-snow-57957dc8d7-rmnd8:/data/coding# cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.4 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
```

NVIDIA官网TensorRT下载链接：https://developer.nvidia.com/nvidia-tensorrt-8x-download，https://developer.nvidia.com/tensorrt/download/10x。

- 解压

```
# 解压到指定目录（如 /opt）
sudo tar -zxvf TensorRT-8.6.1.6.Linux.x86_64-gnu.cuda-12.0.tar.gz -C /data/Trt

# 添加环境变量到 ~/.bashrc
echo 'export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/data/Trt/TensorRT-8.6.1.6/lib' >> ~/.bashrc
echo 'export PATH=$PATH:/data/Trt/TensorRT-8.6.1.6/bin' >> ~/.bashrc
source ~/.bashrc  # 立即生效
```

这里我选择的是8.6的版本，对应的cuda版本是12.0，低于机器上安装的cuda版本12.6。cuda版本具有向下兼容性，因此这里的选择没有问题。

- 安装 Python 接口

```
# 进入解压目录的python子目录，安装对应版本的whl包
cd /data/Trt/TensorRT-8.6.1.6/python
pip install tensorrt-8.6.1-cp310-none-linux_x86_64.whl  # 根据Python版本选择
```

- 添加cuda环境变量

```
echo "/usr/local/cuda/lib64" |  tee /etc/ld.so.conf.d/cuda.conf
ln -s /usr/local/cuda/targets/x86_64-linux/lib/libcublas.so.12 /usr/lib/libcublas.so.12

ln -s /data/Trt/TensorRT-8.6.1.6/lib/libnvinfer.so.8 /usr/lib/libnvinfer.so.8
ln -s /usr/local/cuda/targets/x86_64-linux/lib/libcublasLt.so.12 /usr/lib/libcublasLt.so.12
sudo ldconfig
```

3.2.1.2 安装opencv

```
git clone https://github.com/opencv/opencv.git
#可以使用https://gitcode.com/gh_mirrors/opencv31/opencv.git加速
mkdir build
cd build
cmake -D CMAKE_BUILD_TYPE=Release -D CMAKE_INSTALL_PREFIX=/usr/local ..
make -j4  # 使用 4 个线程进行编译
sudo make install
ln -s /usr/local/lib/libopencv4.so.4.9 /usr/lib/libopencv4.so
sudo ldconfig
#添加到环境变量
echo 'export LD_LIBRARY_PATH=/usr/local/lib:$LD_LIBRARY_PATH' >> ~/.bashrc
source ~/.bashrc
```

验证

```
opencv_version
# 4.13.0-dev
```

3.2.1.3 安装Protobuf

- 解压

```
unzip protobuf-3.11.4.zip
```

- 编译

```
cd protobuf-3.11.4/cmake
cmake . -Dprotobuf_BUILD_TESTS=OFF
cmake --build .
```

- 创建安装目录

我们先要创建一个文件用于存放安装后的protobuf的头文件和库文件，我们选择在`/data`目录下创建一个**protobuf**文件，指令如下：

```
mkdir /data/protobuf
```

- 安装

```
make install DESTDIR=/data/protobuf
```

注意：编译完成后的protobuf文件夹下仅仅只有一个**usr**一个文件夹，需要将编译好的`protobuf/usr/local`下的**bin**、**include**、**lib**文件夹复制到`protobuf`当前文件夹下，方便后续tensorRT_Pro项目的CMakeLists.txt的指定。

```
cp -r /data/protobuf/usr/local/* /data/protobuf
```

- 环境变量的配置

添加如下内容保存并退出(**注意路径修改为自己的路径**)

```
echo 'export PATH=$PATH:/data/protobuf/bin' >> ~/.bashrc
echo 'export PKG_CONFIG_PATH=/data/probobuf/lib/pkgconfig' >> ~/.bashrc
source ~/.bashrc
```

- 配置动态路径

```
echo '/data/protobuf/lib' >> /etc/ld.so.conf
```

- **验证**

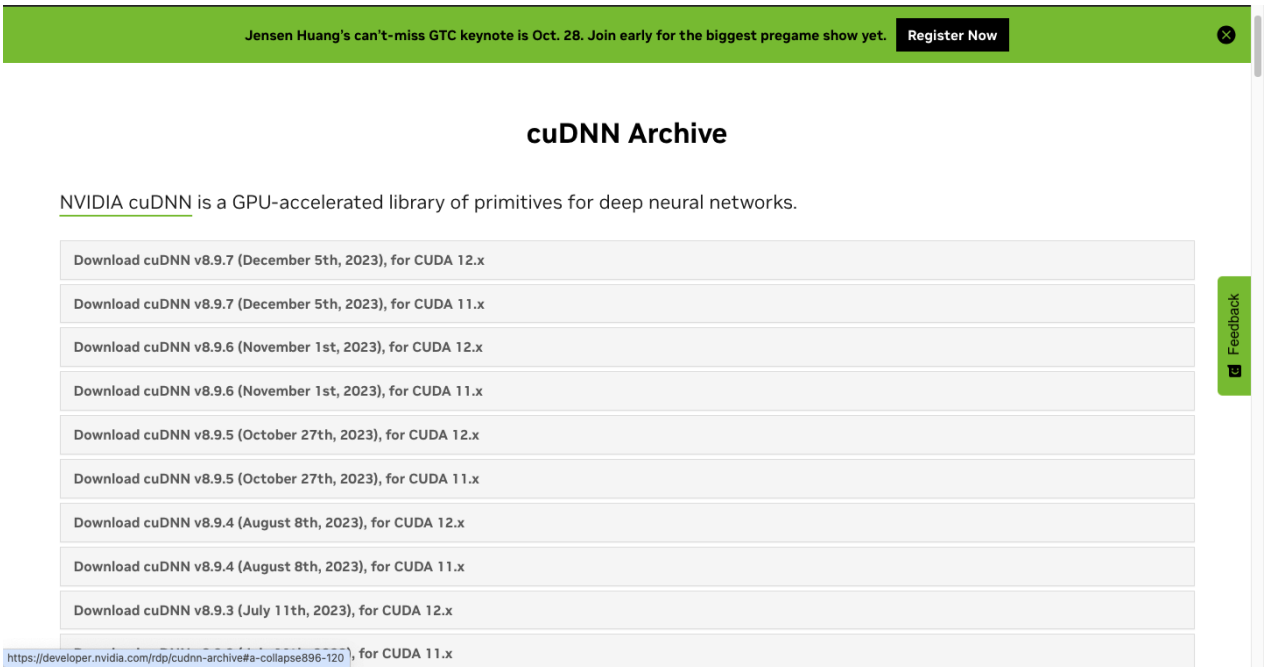
```
protoc --version
# libprotoc 3.11.4
```

3.2.1.4 安装cudnn

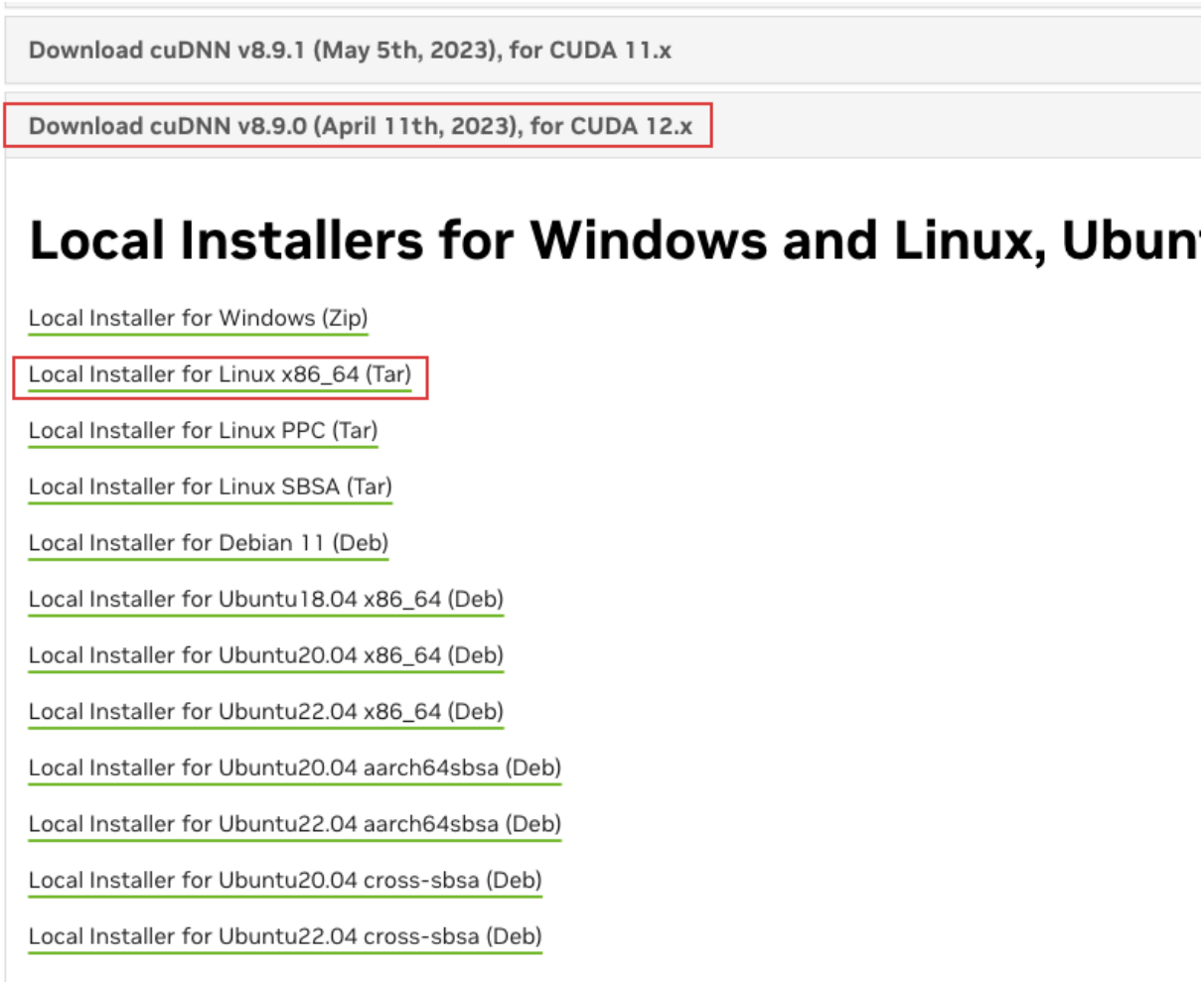
cuDNN的版本选择是根据CUDA版本来的，在前面CUDA的安装中，我们选择的是CUDA 12.6 版本，依据此我们来安装cuDNN

> NVIDIA官网cuDNN下载链接：https://developer.nvidia.com/rdp/cudnn-archive

打开上面的链接，会出现如下的界面:



根据你的CUDA版本选择对应的cuDNN即可，CUDA 12.x代表CUDA12版本的都支持，这里我选择的是Download cuDNN v8.9.0 (April 11th, 2023), for CUDA 12.x，如下所示，点击之后选择对应的平台安装包下载就行，如下面红色框所示，选择的是Linux，Ubuntu，x86_64的Tar安装包



- 解压

```
tar -xf cudnn-linux-x86_64-8.9.0.131_cuda12-archive.tar.xz
```

- 复制文件

等待解压完成，解压完成之后在目录下有一个cudnn-linux-x86_64-8.4.0.27_cuda11.6-archive文件夹，这个文件夹中又包含include和lib两个文件夹，分别代表cuDNN头文件和cuDNN库文件。

cuDNN非常简单，就是将这两个文件中的所有内容复制到CUDA对应的文件夹中

```
cp /data/cudnn-linux-x86_64-8.9.0.131_cuda12-archive/lib/*
/usr/local/cuda/lib64
cp /data/cudnn-linux-x86_64-8.9.0.131_cuda12-archive/include/*
/usr/local/cuda/include
```

3.2.2 修改代码

3.2.2.1 配置CMakeLists.txt

进入项目tensorRT_Pro-YOLOv8，tensorRT_Pro-YOLOv8 提供 CMakeLists.txt 和 Makefile 两种编译方式，**二者选一即可**，这里我们选择使用CMake方式。

主要修改五处：

1. 修改第 13 行，修改 OpenCV 路径

```
set(OpenCV_DIR   "/usr/local/include/opencv4")
```

2. 修改第 15 行，修改 CUDA 路径

```
set(CUDA_TOOLKIT_ROOT_DIR     "/usr/local/cuda")
```

3. 修改第 16 行，修改 cuDNN 路径

```
set(CUDNN_DIR    "/data/cudnn-linux-x86_64-8.9.0.131_cuda12-archive")
```

4. 修改第 17 行，修改 tensorRT路径

```
set(TENSORRT_DIR "/data/Trt/TensorRT-8.6.1.6")
```

5. 修改第 20 行，修改 protobuf 路径

```
set(PROTOBUF_DIR "/data/protobuf/user/local")
```

6. 修改第 5 行，将c++的版本修改为14

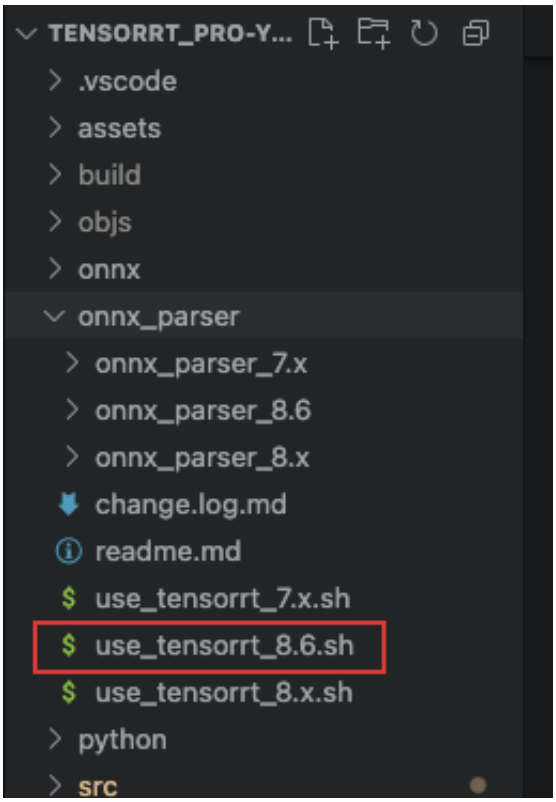
```
set(CMAKE_CXX_STANDARD 14)  # 添加这一行
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

7. 修改第 10 行，取消注释，修改显卡计算能力。我这里的显卡是rtx3060，计算能力是sm_86

```
set(CUDA_GEN_CODE "-gencode=arch=compute_86,code=sm_86")
```

3.2.2.2 配置onnx_parser版本

由于我们安装的是Tensorrt 8.6版本，需要在项目中配置使用onnx_parser_8.6



在项目目录下执行如下命令：

```
sh onnx_parser/use_tensorrt_8.6.sh
```

3.2.2.3 修改onnx文件名

在文件src/application/app_yolo.cpp中修改：

```
int app_yolo(){

    // test(Yolo::Type::V8, TRT::Mode::FP32, "yolov8s");
    // test_single_image();
    // test_video();
    // test(Yolo::Type::V3, TRT::Mode::FP32, "yolov3");
    // test(Yolo::Type::X, TRT::Mode::FP32, "yolox_s");
    // test(Yolo::Type::V5, TRT::Mode::FP32, "yolov5s");
    // test(Yolo::Type::V6, TRT::Mode::FP32, "yolov6s");
    // test(Yolo::Type::V7, TRT::Mode::FP32, "yolov7");
    // test(Yolo::Type::V9, TRT::Mode::FP32, "yolov9c");
    // test(Yolo::Type::V10, TRT::Mode::FP32, "yolov10s");
    test(Yolo::Type::V11, TRT::Mode::FP32, "best");
    // test(Yolo::Type::V12, TRT::Mode::FP32, "yolov12s");
    // test(Yolo::Type::V13, TRT::Mode::FP32, "yolov13s");
```

```
    return 0;
}
```

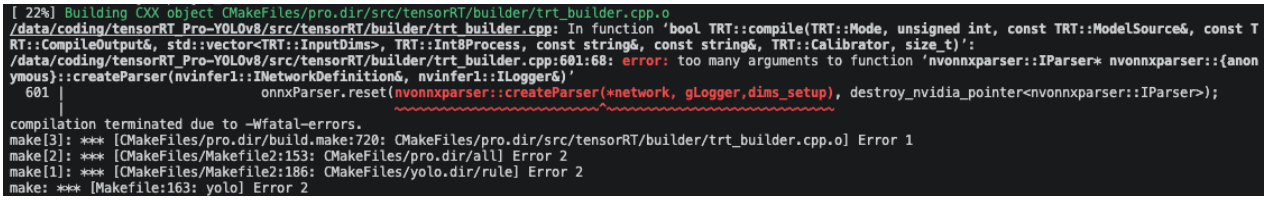
`best`为我们上传到workspace的onnx文件名。

3.3 编译

将best.onnx上传到项目workspace目录下。然后终端执行如下命令：

```
mkdir build
cd build
cmake ..
make yolo -j64
```

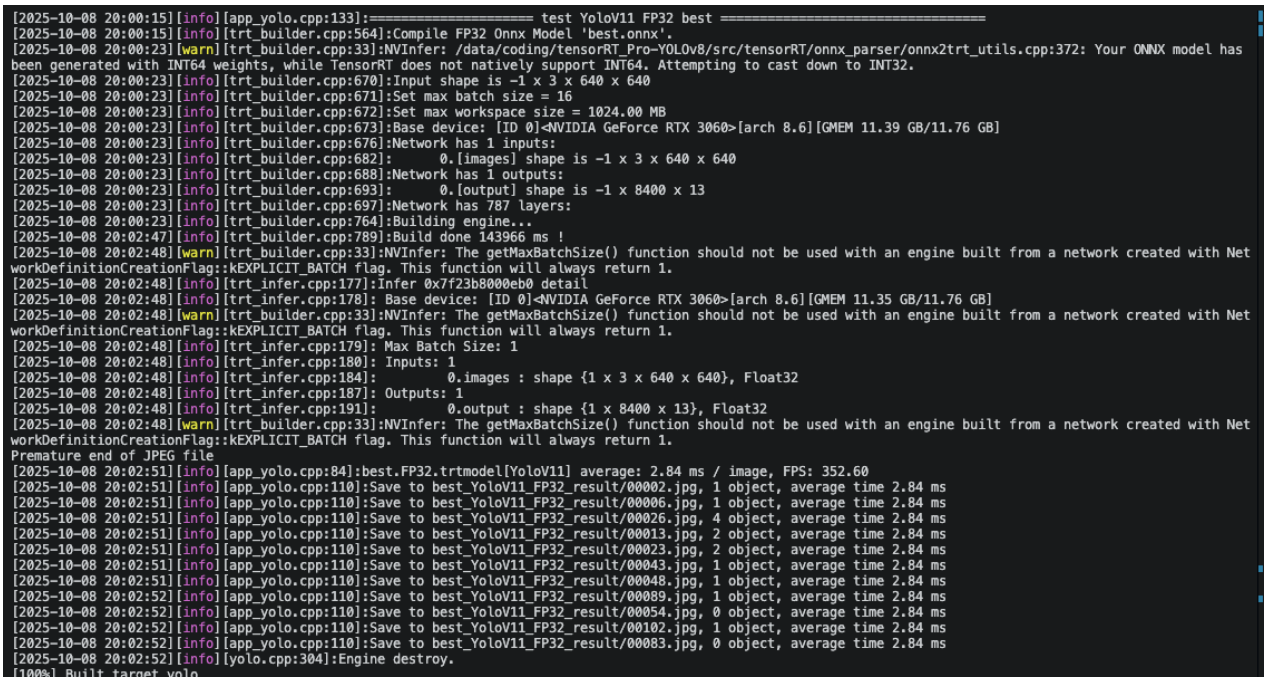
发现如下报错信息：



这是因为项目代码是使用较低版本的tensorrt的api编写的，进入src/tensorRT/builder/trt_builder.cpp文件，修改第601行代码：

```
// onnxParser.reset(nvonnxparser::createParser(*network, gLogger,dims_setup), destroy_nvidia_pointer<nvonnxparser::IParser>);
onnxParser.reset(nvonnxparser::createParser(*network, gLogger), destroy_nvidia_pointer<nvonnxparser::IParser>);
```

再次编译，程序成功运行，结果如下：



编译运行成功后在 workspace 文件夹下会生成 engine 文件 **best.FP32.trtmodel** 用于模型推理，同时它还会生成 **best_YoloV11_FP32_result** 文件夹，该文件夹下保存了推理的图片。

3.4 前后处理

3.4.1 前处理

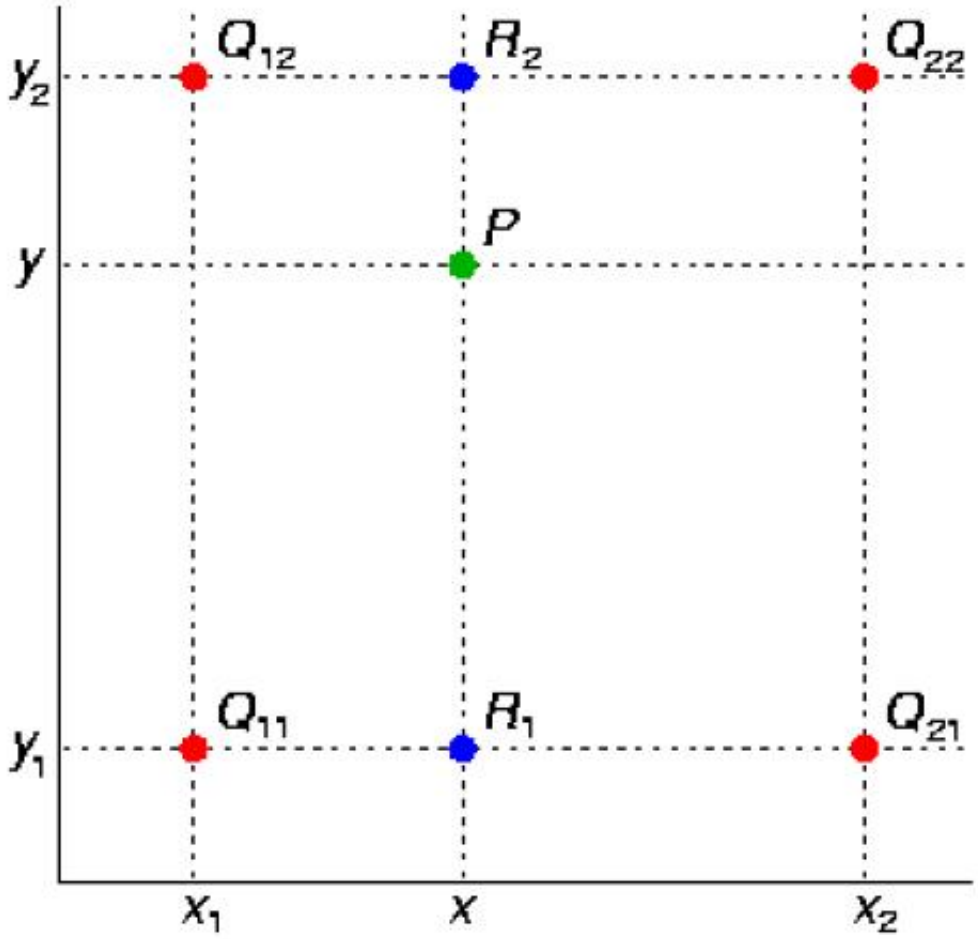


图 1

双线性插值：https://blog.csdn.net/qq_30150579/article/details/140812554

tensorRT_Pro 中预处理的代码(src/tensorRT/common/preprocess_kernel.cu)如下：

```
  /**
   * @brief 对输入图像进行仿射变换、双线性插值和归一化处理，输出为平面格式的浮点型数据
   *
   * 该核函数实现了以下功能：
   * 1. 根据给定的 2x3 仿射变换矩阵，计算目标图像每个像素点在源图像中的对应位置
   * 2. 使用双线性插值法从源图像中获取该位置的像素值
   * 3. 如果计算出的源图像坐标超出边界，则使用指定的常数值填充
   * 4. 对插值得到的像素值进行归一化处理（支持均值标准差归一化或 alpha-beta 变换）
   * 5. 支持通道顺序反转（RGB <-> BGR）
   * 6. 输出为平面格式（planar format）：所有 R 值连续存储，然后是所有 G 值，最后是所有 B 值
   *
   * @param src 指向源图像数据的指针（uint8_t 类型，BGR 格式）
   * @param src_line_size 源图像每行的字节数（可能包含对齐填充）
   * @param src_width 源图像宽度（像素）
   * @param src_height 源图像高度（像素）
   * @param dst 指向目标图像数据的指针（float 类型，平面格式）
```

```
    * @param dst_width 目标图像宽度（像素）
    * @param dst_height 目标图像高度（像素）
    * @param const_value_st 当源坐标越界时用于填充的常数值
    * @param warp_affine_matrix_2_3 指向 2x3 仿射变换矩阵的指针（按行优先顺序存储）
    * @param norm 归一化参数结构体，包含归一化类型、均值、标准差、alpha、beta 等
    * @param edge 线程处理的总像素数，用于边界检查
    */
   __global__ void warp_affine_bilinear_and_normalize_plane_kernel(
     uint8_t* src, int src_line_size, int src_width, int src_height,
     float* dst, int dst_width, int dst_height,
     uint8_t const_value_st, float* warp_affine_matrix_2_3, Norm norm, int edge){

     // 计算当前线程负责处理的像素位置
     int position = blockDim.x * blockIdx.x + threadIdx.x;
     if (position >= edge) return;

     // 从仿射变换矩阵中提取参数
     // 第一行: [m_x1, m_y1, m_z1]
     // 第二行: [m_x2, m_y2, m_z2]
     float m_x1 = warp_affine_matrix_2_3[0];
     float m_y1 = warp_affine_matrix_2_3[1];
     float m_z1 = warp_affine_matrix_2_3[2];
     float m_x2 = warp_affine_matrix_2_3[3];
     float m_y2 = warp_affine_matrix_2_3[4];
     float m_z2 = warp_affine_matrix_2_3[5];

     // 计算目标图像中的像素坐标 (dx, dy)
     int dx      = position % dst_width;
     int dy      = position / dst_width;

     // 应用仿射变换计算源图像中的浮点坐标 (src_x, src_y)
     float src_x = m_x1 * dx + m_y1 * dy + m_z1;
     float src_y = m_x2 * dx + m_y2 * dy + m_z2;
     float c0, c1, c2;

     // 边界检查：如果源坐标在有效范围外，则使用常数值填充
     if(src_x <= -1 || src_x >= src_width || src_y <= -1 || src_y >= src_height){
       // out of range
       c0 = const_value_st;
       c1 = const_value_st;
       c2 = const_value_st;
     }else{
       // 双线性插值计算像素值
       // 找到包围 (src_x, src_y) 的四个整数坐标点
       int y_low = floorf(src_y);
       int x_low = floorf(src_x);
```

```
      int y_high = y_low + 1;
      int x_high = x_low + 1;

      // 定义边界外的默认填充值
      uint8_t const_value[] = {const_value_st, const_value_st,
const_value_st};

      // 计算插值权重
      float ly    = src_y - y_low;  // y 方向的小数部分
      float lx    = src_x - x_low;  // x 方向的小数部分
      float hy    = 1 - ly;         // y 方向的补集
      float hx    = 1 - lx;         // x 方向的补集
      float w1    = hy * hx;        // 左上角权重
      float w2    = hy * lx;        // 右上角权重
      float w3    = ly * hx;        // 左下角权重
      float w4    = ly * lx;        // 右下角权重

      // 初始化四个角点的像素值为默认填充值
      uint8_t* v1 = const_value;
      uint8_t* v2 = const_value;
      uint8_t* v3 = const_value;
      uint8_t* v4 = const_value;

      // 如果角点在图像范围内，则更新为实际像素值
      // 左上角 (y_low, x_low) 和右上角 (y_low, x_high)
      if(y_low >= 0){
        if (x_low >= 0)
          v1 = src + y_low * src_line_size + x_low * 3;

        if (x_high < src_width)
          v2 = src + y_low * src_line_size + x_high * 3;
      }

      // 左下角 (y_high, x_low) 和右下角 (y_high, x_high)
      if(y_high < src_height){
        if (x_low >= 0)
          v3 = src + y_high * src_line_size + x_low * 3;

        if (x_high < src_width)
          v4 = src + y_high * src_line_size + x_high * 3;
      }

      // 使用双线性插值公式计算最终的像素值，并四舍五入
      // same to opencv
      c0 = floorf(w1 * v1[0] + w2 * v2[0] + w3 * v3[0] + w4 * v4[0] +
0.5f);
      c1 = floorf(w1 * v1[1] + w2 * v2[1] + w3 * v3[1] + w4 * v4[1] +
0.5f);
```

```
      c2 = floorf(w1 * v1[2] + w2 * v2[2] + w3 * v3[2] + w4 * v4[2] + 0.5f);
    }

    // 根据通道类型决定是否反转通道顺序（RGB <-> BGR）
    if(norm.channel_type == ChannelType::Invert){
      float t = c2;
      c2 = c0;  c0 = t;
    }

    // 根据归一化类型对像素值进行归一化处理
    if(norm.type == NormType::MeanStd){
      // 均值标准差归一化: (pixel * alpha - mean) / std
      c0 = (c0 * norm.alpha - norm.mean[0]) / norm.std[0];
      c1 = (c1 * norm.alpha - norm.mean[1]) / norm.std[1];
      c2 = (c2 * norm.alpha - norm.mean[2]) / norm.std[2];
    }else if(norm.type == NormType::AlphaBeta){
      // Alpha-Beta 变换: pixel * alpha + beta
      c0 = c0 * norm.alpha + norm.beta;
      c1 = c1 * norm.alpha + norm.beta;
      c2 = c2 * norm.alpha + norm.beta;
    }
    // 如果归一化类型为 None，则不进行任何处理

    // 计算目标图像的总面积（用于平面格式的偏移计算）
    int area = dst_width * dst_height;

    // 计算目标图像中当前像素的三个通道值的存储位置
    // 平面格式：所有 R 值连续存储，然后是所有 G 值，最后是所有 B 值
    float* pdst_c0 = dst + dy * dst_width + dx;           // R 通道
    float* pdst_c1 = pdst_c0 + area;                      // G 通道
    float* pdst_c2 = pdst_c1 + area;                      // B 通道

    // 将处理后的像素值写入目标图像
    *pdst_c0 = c0;
    *pdst_c1 = c1;
    *pdst_c2 = c2;
  }
```

在 CUDA 中我们是对每个像素进行操作，因此非常容易实现 BGR → RGB，/255.0 等操作。关于代码的具体分析可以参考 YOLOv5推理详解及预处理高性能实现，

3.4.2 后处理

位于文件：src/application/app_yolo/yolo_decode.cu

- warp_affine

```
/**
     * @brief CUDA核函数，用于YOLOv8模型的边界框解码
```

```
     *
     * 该函数负责将YOLOv8模型的预测输出转换为图像坐标系中的边界框，
     * 并根据置信度阈值过滤预测结果。
     * 1,8400,13
     * @param predict 指向模型预测结果的指针，格式为 [x, y, w, h, class_confidence...]
     * @param num_bboxes 预测的边界框总数
     * @param num_classes 分类数量
     * @param confidence_threshold 置信度阈值，低于此值的预测将被过滤
     * @param invert_affine_matrix 逆仿射变换矩阵，用于将坐标转换回原始图像空间
     * @param parray 输出数组，用于存储解码后的边界框信息
     * @param MAX_IMAGE_BOXES 每张图像允许的最大边界框数量
     */
    static __global__ void decode_kernel_v8(
        float *predict,
        int num_bboxes,
        int num_classes,
        float confidence_threshold,
        float* invert_affine_matrix,
        float* parray,
        int MAX_IMAGE_BOXES
    ){

        // 计算当前线程处理的边界框索引
        int position = blockDim.x * blockIdx.x + threadIdx.x;
        // 边界检查，超出范围直接返回
        if (position >= num_bboxes) return;

        // 定位到当前边界框的预测数据起始位置
        // YOLOv8格式：[cx, cy, width, height, class0_conf, class1_conf, ...]
        float* pitem = predict + (4 + num_classes) * position;

        // 指向类别置信度数据的起始位置
        float* class_confidence = pitem + 4;

        // 初始化最高类别置信度和对应标签
        float confidence = *class_confidence++;  // 第0类的置信度
        int label = 0;

        // 遍历所有类别，找到置信度最高的类别
        for(int i = 1; i < num_classes; ++i, ++class_confidence){
            if(*class_confidence > confidence){
                confidence = *class_confidence;
                label = i;
            }
        }

        // 如果最高类别置信度低于阈值，则过滤该预测
```

```
        if(confidence < confidence_threshold)
            return;

        // 原子操作获取输出数组中的索引位置
        int index = atomicAdd(parray, 1);
        // 如果超出最大边界框数量限制，则丢弃
        if(index >= MAX_IMAGE_BOXES)
            return;

        // 提取边界框中心坐标和宽高
        float cx = *pitem++;     // 中心x坐标
        float cy = *pitem++;     // 中心y坐标
        float width = *pitem++;  // 宽度
        float height = *pitem++; // 高度

        // 从中心坐标格式转换为左上右下坐标格式
        float left   = cx - width  * 0.5f;   // 左边界
        float top    = cy - height * 0.5f;   // 上边界
        float right  = cx + width  * 0.5f;   // 右边界
        float bottom = cy + height * 0.5f;   // 下边界

        // 应用逆仿射变换，将坐标转换回原始图像空间
        affine_project(invert_affine_matrix, left,  top,    &left,  &top);
        affine_project(invert_affine_matrix, right, bottom, &right,
&bottom);

        // 计算输出数组中当前边界框的存储位置
        // parray[0]存储边界框计数，实际数据从parray[1]开始
        float *pout_item = parray + 1 + index * NUM_BOX_ELEMENT;

        // 存储边界框信息：left, top, right, bottom, confidence, label,
keepflag
        *pout_item++ = left;      // 左边界
        *pout_item++ = top;       // 上边界
        *pout_item++ = right;     // 右边界
        *pout_item++ = bottom;    // 下边界
        *pout_item++ = confidence; // 置信度
        *pout_item++ = label;     // 类别标签
        *pout_item++ = 1;         // 保留标志：1=保留，0=忽略
    }

```

- nms

```
   /**
     * @brief CUDA核函数，用于执行非极大值抑制(NMS)算法
     *
     * 该函数通过比较边界框之间的IoU(交并比)来过滤重叠的预测框，
     * 保留置信度较高的边界框，去除冗余的预测结果。
```

```
     *
     * @param bboxes 指向边界框数组的指针，格式为 [count, box1_data, box2_data, ...]
     *               其中每个box_data包含 [left, top, right, bottom, confidence, class, keepflag]
     * @param max_objects 允许的最大边界框数量
     * @param threshold IoU阈值，超过此值的重叠框将被抑制
     */
    static __global__ void nms_kernel(float* bboxes, int max_objects, float threshold){

        // 计算当前线程处理的边界框索引
        int position = (blockDim.x * blockIdx.x + threadIdx.x);

        // 获取有效的边界框数量，不超过最大限制
        int count = min((int)*bboxes, max_objects);

        // 边界检查，超出有效范围直接返回
        if (position >= count)
            return;

        // 定位到当前处理的边界框数据起始位置
        // bboxes[0]存储边界框计数，实际数据从bboxes[1]开始
        // 每个边界框包含NUM_BOX_ELEMENT个元素
        float* pcurrent = bboxes + 1 + position * NUM_BOX_ELEMENT;

        // 遍历所有边界框，与当前框进行比较
        for(int i = 0; i < count; ++i){
            // 定位到第i个边界框的数据起始位置
            float* pitem = bboxes + 1 + i * NUM_BOX_ELEMENT;

            // 跳过自身比较或不同类别的情况
            if(i == position || pcurrent[5] != pitem[5]) continue;

            // 只有当比较框的置信度大于等于当前框时才进行IoU计算
            // 相同置信度时，保留索引较小的框
            if(pitem[4] >= pcurrent[4]){
                // 置信度相同时，保留索引较小的框
                if(pitem[4] == pcurrent[4] && i < position)
                    continue;

                // 计算两个边界框之间的IoU
                float iou = box_iou(
                    pcurrent[0], pcurrent[1], pcurrent[2], pcurrent[3],  // 当前框坐标
                    pitem[0],    pitem[1],    pitem[2],    pitem[3]     // 比较框坐标
                );

```

```
                // 如果IoU超过阈值，则抑制当前框（标记为忽略）
                if(iou > threshold){
                    pcurrent[6] = 0;  // 0=ignore, 1=keep
                    return;
                }
            }
        }
    }
```

3.5 推理优化

3.5.1 onnx简化

`onnx-simplifier`（`onnxsim`），它是一个用于简化 ONNX 模型的 Python 工具，可以优化模型结构、删除冗余节点，并提高模型在推理框架（如 TensorRT）中的兼容性。

- 安装

```
pip install onnxsim onnxruntime
```

- 在workspace目录下新建文件onnx_simplify.py，并编写如下代码：

```
import onnx
from onnxsim import simplify

# 加载原始 ONNX 模型
input_path = "best.onnx"
output_path = "simplified_model.onnx"

# 简化模型
model = onnx.load(input_path)
simplified_model, check = simplify(model)  # check=True 会验证简化后的模型

# 保存简化后的模型
onnx.save(simplified_model, output_path)
print(f"模型已简化并保存到: {output_path}")
```

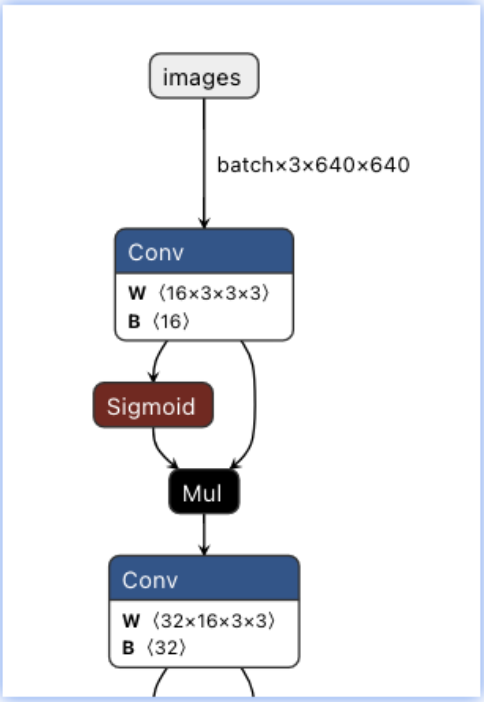
- 运行

```
python onnx_simplify.py
```

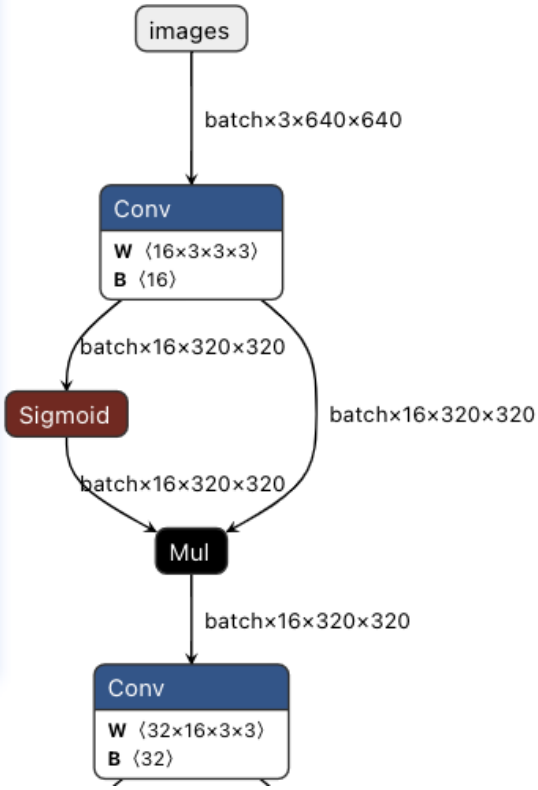
- 对比优化效果

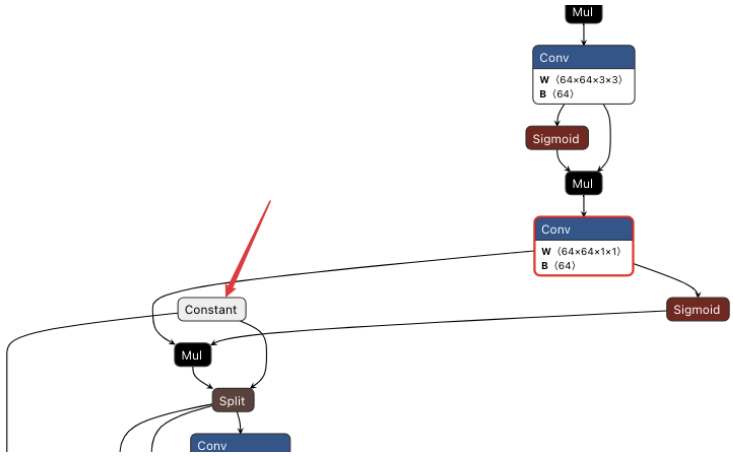
使用 Netron 可视化工具分别查看简化前后的计算图：

前

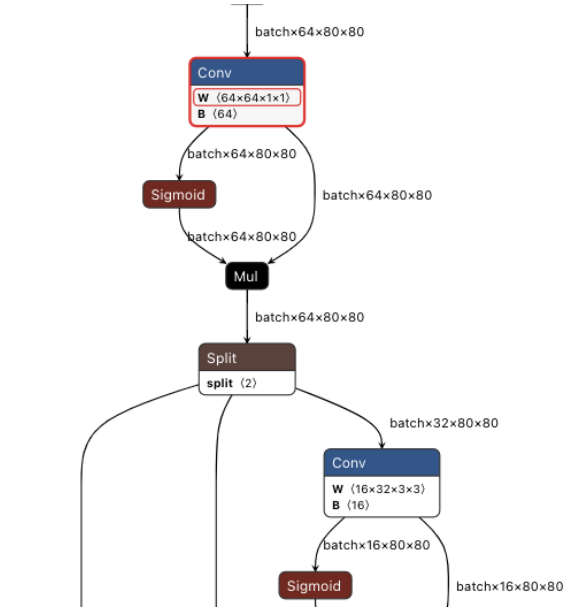

images
batch×3×640×640
Conv
W ⟨16×3×3×3⟩
B ⟨16⟩
Sigmoid
Mul
Conv
W ⟨32×16×3×3⟩
B ⟨32⟩

后

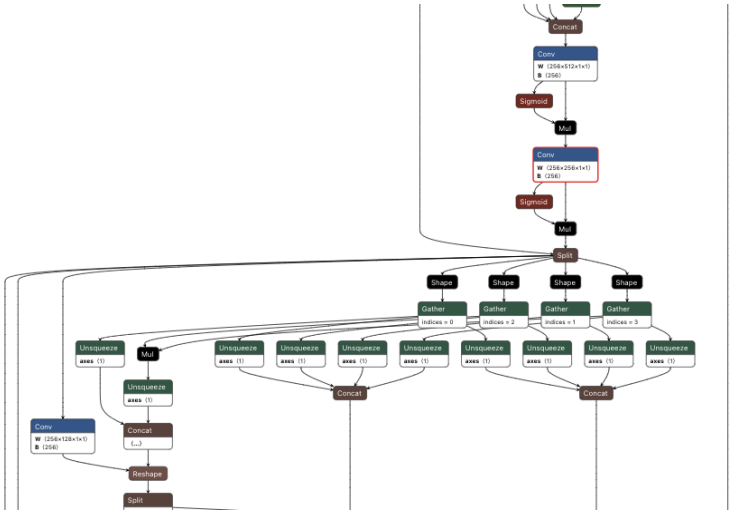

images
batch×3×640×640
Conv
W ⟨16×3×3×3⟩
B ⟨16⟩
batch×16×320×320
Sigmoid
batch×16×320×320
batch×16×320×320
Mul
batch×16×320×320
Conv
W ⟨32×16×3×3⟩
B ⟨32⟩


Mul
Conv
W ⟨64×64×3×3⟩
B ⟨64⟩
Sigmoid
Mul
Conv
W ⟨64×64×1×1⟩
B ⟨64⟩
Constant
Sigmoid
Mul
Split
Conv


type Conv
module ai.onnx v11
name /model.4/cv1/conv/Conv
ATTRIBUTES
dilations 1, 1
group 1
kernel_shape 1, 1
pads 0, 0, 0, 0
strides 1, 1
INPUTS
X name: /model.3/act/Mul_output_0
W name: model.4.cv1.conv.weight
B name: model.4.cv1.conv.bias


batch×64×80×80
Conv
W ⟨64×64×1×1⟩
B ⟨64⟩
batch×64×80×80
Sigmoid
batch×64×80×80
batch×64×80×80
Mul
batch×64×80×80
Split
split ⟨2⟩
batch×32×80×80
Conv
W ⟨16×32×3×3⟩
B ⟨16⟩
batch×16×80×80
Sigmoid
batch×16×80×80


type Conv
module ai.onnx v11
name /model.4/cv1/conv/Conv
ATTRIBUTES
dilations 1, 1
group 1
kernel_shape 1, 1
pads 0, 0, 0, 0
strides 1, 1
INPUTS
X name: /model.3/act/Mul_output_0
W name: model.4.cv1.conv.weight
B name: model.4.cv1.conv.bias
OUTPUTS
Y name: /model.4/cv1/conv/Conv_output_0



type Conv
module ai.onnx v11
name /model.10/cv1/conv/Conv

ATTRIBUTES

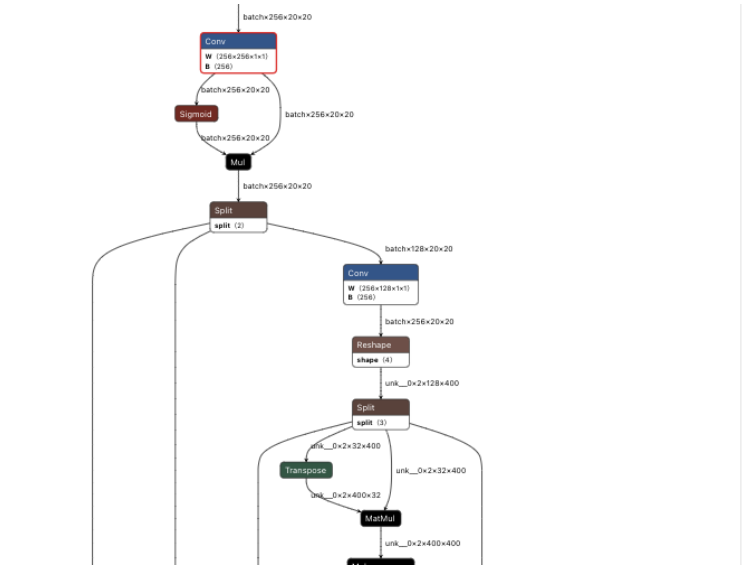
dilations 1, 1
group 1
kernel_shape 1, 1
pads 0, 0, 0, 0
strides 1, 1

INPUTS

X name: /model.9/cv2/act/Mul_output_0
W name: model.10.cv1.conv.weight
B name: model.10.cv1.conv.bias

OUTPUTS

Y name: /model.10/cv1/conv/Conv_output_0



type Conv
module ai.onnx v11
name /model.10/cv1/conv/Conv

ATTRIBUTES

dilations 1, 1
group 1
kernel_shape 1, 1
pads 0, 0, 0, 0
strides 1, 1

INPUTS

X name: /model.9/cv2/act/Mul_output_0
W name: model.10.cv1.conv.weight
B name: model.10.cv1.conv.bias

OUTPUTS

Y name: /model.10/cv1/conv/Conv_output_0

- 重新编译项目

修改app_yolo.cpp中的模型文件名称

```
test(Yolo::Type::V11, TRT::Mode::FP32, "simplified_model");
```

```
cd build
make yolo -j64
```

结果如下：

```
[100%] Built target pro
[2025-10-06 15:12:32][info][app_yolo.cpp:129]:==================== test YoloV11 FP32 simplified_model ====================
[2025-10-06 15:12:32][info][trt_builder.cpp:564]:Compile FP32 Onnx Model 'simplified_model.onnx'.
[2025-10-06 15:12:40][warn][trt_builder.cpp:33]:NVInfer: /data/coding/tensorRT_Pro-YOLOv8/src/tensorRT/onnx_parser/onnx2trt_utils.cpp:372: Your ONNX model has
been generated with INT64 weights, while TensorRT does not natively support INT64. Attempting to cast down to INT32.
[2025-10-06 15:12:40][info][trt_builder.cpp:671]:Input shape is -1 x 3 x 640 x 640
[2025-10-06 15:12:40][info][trt_builder.cpp:672]:Set max batch size = 16
[2025-10-06 15:12:40][info][trt_builder.cpp:673]:Set max workspace size = 1024.00 MB
[2025-10-06 15:12:40][info][trt_builder.cpp:674]:Base device: [ID 0]<NVIDIA GeForce RTX 3060>[arch 8.6][GMEM 11.39 GB/11.76 GB]
[2025-10-06 15:12:40][info][trt_builder.cpp:677]:Network has 1 inputs:
[2025-10-06 15:12:40][info][trt_builder.cpp:683]:      0.[images] shape is -1 x 3 x 640 x 640
[2025-10-06 15:12:40][info][trt_builder.cpp:689]:Network has 1 outputs:
[2025-10-06 15:12:40][info][trt_builder.cpp:694]:      0.[output] shape is -1 x 8400 x 13
[2025-10-06 15:12:40][info][trt_builder.cpp:698]:Network has 510 layers:
[2025-10-06 15:12:40][info][trt_builder.cpp:765]:Building engine...
[2025-10-06 15:15:01][info][trt_builder.cpp:790]:Build done 141357 ms !
[2025-10-06 15:15:02][warn][trt_builder.cpp:33]:NVInfer: The getMaxBatchSize() function should not be used with an engine built from a network created with Ne
workDefinitionCreationFlag::kEXPLICIT_BATCH flag. This function will always return 1.
[2025-10-06 15:15:02][info][trt_infer.cpp:177]:Infer 0x7f914c000ec0 detail
[2025-10-06 15:15:02][info][trt_infer.cpp:178]: Base device: [ID 0]<NVIDIA GeForce RTX 3060>[arch 8.6][GMEM 11.35 GB/11.76 GB]
[2025-10-06 15:15:02][warn][trt_builder.cpp:33]:NVInfer: The getMaxBatchSize() function should not be used with an engine built from a network created with Ne
workDefinitionCreationFlag::kEXPLICIT_BATCH flag. This function will always return 1.
[2025-10-06 15:15:02][info][trt_infer.cpp:179]: Max Batch Size: 1
[2025-10-06 15:15:02][info][trt_infer.cpp:180]: Inputs: 1
[2025-10-06 15:15:02][info][trt_infer.cpp:184]:         0.images : shape {1 x 3 x 640 x 640}, Float32
[2025-10-06 15:15:02][info][trt_infer.cpp:187]: Outputs: 1
[2025-10-06 15:15:02][info][trt_infer.cpp:191]:         0.output : shape {1 x 8400 x 13}, Float32
[2025-10-06 15:15:02][warn][trt_builder.cpp:33]:NVInfer: The getMaxBatchSize() function should not be used with an engine built from a network created with Ne
workDefinitionCreationFlag::kEXPLICIT_BATCH flag. This function will always return 1.
[2025-10-06 15:15:04][info][app_yolo.cpp:80]:simplified_model.FP32.trtmodel[YoloV11] average: 2.78 ms / image, FPS: 360.22
[2025-10-06 15:15:04][info][app_yolo.cpp:106]:Save to simplified_model_YoloV11_FP32_result/00002.jpg, 1 object, average time 2.78 ms
[2025-10-06 15:15:04][info][app_yolo.cpp:106]:Save to simplified_model_YoloV11_FP32_result/00006.jpg, 1 object, average time 2.78 ms
[2025-10-06 15:15:04][info][app_yolo.cpp:106]:Save to simplified_model_YoloV11_FP32_result/00013.jpg, 2 object, average time 2.78 ms
[2025-10-06 15:15:04][info][app_yolo.cpp:106]:Save to simplified_model_YoloV11_FP32_result/00023.jpg, 2 object, average time 2.78 ms
[2025-10-06 15:15:04][info][app_yolo.cpp:106]:Save to simplified_model_YoloV11_FP32_result/00026.jpg, 4 object, average time 2.78 ms
[2025-10-06 15:15:04][info][app_yolo.cpp:106]:Save to simplified_model_YoloV11_FP32_result/00043.jpg, 1 object, average time 2.78 ms
[2025-10-06 15:15:04][info][yolo.cpp:304]:Engine destroy.
[100%] Built target yolo
```

对比优化前：

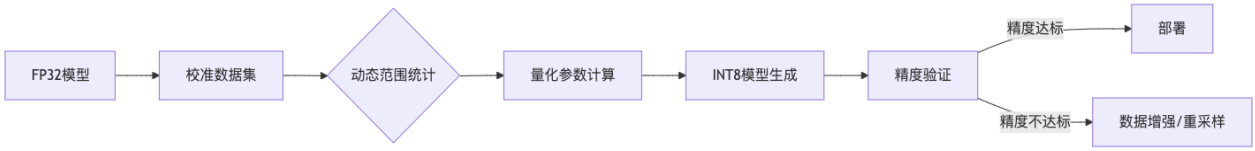
```
[2025-10-08 20:00:15][info][app_yolo.cpp:133]:==================== test YoloV11 FP32 best ====================
[2025-10-08 20:00:15][info][trt_builder.cpp:564]:Compile FP32 Onnx Model 'best.onnx'.
[2025-10-08 20:00:23][warn][trt_builder.cpp:33]:NVInfer: /data/coding/tensorRT_Pro-YOLOv8/src/tensorRT/onnx_parser/onnx2trt_utils.cpp:372: Your ONNX model has
been generated with INT64 weights, while TensorRT does not natively support INT64. Attempting to cast down to INT32.
[2025-10-08 20:00:23][info][trt_builder.cpp:670]:Input shape is -1 x 3 x 640 x 640
[2025-10-08 20:00:23][info][trt_builder.cpp:671]:Set max batch size = 16
[2025-10-08 20:00:23][info][trt_builder.cpp:672]:Set max workspace size = 1024.00 MB
[2025-10-08 20:00:23][info][trt_builder.cpp:673]:Base device: [ID 0]<NVIDIA GeForce RTX 3060>[arch 8.6][GMEM 11.39 GB/11.76 GB]
[2025-10-08 20:00:23][info][trt_builder.cpp:676]:Network has 1 inputs:
[2025-10-08 20:00:23][info][trt_builder.cpp:682]:      0.[images] shape is -1 x 3 x 640 x 640
[2025-10-08 20:00:23][info][trt_builder.cpp:688]:Network has 1 outputs:
[2025-10-08 20:00:23][info][trt_builder.cpp:693]:      0.[output] shape is -1 x 8400 x 13
[2025-10-08 20:00:23][info][trt_builder.cpp:697]:Network has 787 layers:
[2025-10-08 20:00:23][info][trt_builder.cpp:764]:Building engine...
[2025-10-08 20:02:47][info][trt_builder.cpp:789]:Build done 143966 ms !
[2025-10-08 20:02:48][warn][trt_builder.cpp:33]:NVInfer: The getMaxBatchSize() function should not be used with an engine built from a network created with Net
workDefinitionCreationFlag::kEXPLICIT_BATCH flag. This function will always return 1.
[2025-10-08 20:02:48][info][trt_infer.cpp:177]:Infer 0x7f23b8000eb0 detail
[2025-10-08 20:02:48][info][trt_infer.cpp:178]: Base device: [ID 0]<NVIDIA GeForce RTX 3060>[arch 8.6][GMEM 11.35 GB/11.76 GB]
[2025-10-08 20:02:48][warn][trt_builder.cpp:33]:NVInfer: The getMaxBatchSize() function should not be used with an engine built from a network created with Net
workDefinitionCreationFlag::kEXPLICIT_BATCH flag. This function will always return 1.
[2025-10-08 20:02:48][info][trt_infer.cpp:179]: Max Batch Size: 1
[2025-10-08 20:02:48][info][trt_infer.cpp:180]: Inputs: 1
[2025-10-08 20:02:48][info][trt_infer.cpp:184]:         0.images : shape {1 x 3 x 640 x 640}, Float32
[2025-10-08 20:02:48][info][trt_infer.cpp:187]: Outputs: 1
[2025-10-08 20:02:48][info][trt_infer.cpp:191]:         0.output : shape {1 x 8400 x 13}, Float32
[2025-10-08 20:02:48][warn][trt_builder.cpp:33]:NVInfer: The getMaxBatchSize() function should not be used with an engine built from a network created with Net
workDefinitionCreationFlag::kEXPLICIT_BATCH flag. This function will always return 1.
Premature end of JPEG file
[2025-10-08 20:02:51][info][app_yolo.cpp:84]:best.FP32.trtmodel[YoloV11] average: 2.84 ms / image, FPS: 352.60
[2025-10-08 20:02:51][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00002.jpg, 1 object, average time 2.84 ms
[2025-10-08 20:02:51][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00006.jpg, 1 object, average time 2.84 ms
[2025-10-08 20:02:51][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00026.jpg, 4 object, average time 2.84 ms
[2025-10-08 20:02:51][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00013.jpg, 2 object, average time 2.84 ms
[2025-10-08 20:02:51][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00023.jpg, 2 object, average time 2.84 ms
[2025-10-08 20:02:51][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00043.jpg, 1 object, average time 2.84 ms
[2025-10-08 20:02:51][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00048.jpg, 1 object, average time 2.84 ms
[2025-10-08 20:02:52][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00089.jpg, 1 object, average time 2.84 ms
[2025-10-08 20:02:52][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00054.jpg, 0 object, average time 2.84 ms
[2025-10-08 20:02:52][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00102.jpg, 1 object, average time 2.84 ms
[2025-10-08 20:02:52][info][app_yolo.cpp:110]:Save to best_YoloV11_FP32_result/00083.jpg, 0 object, average time 2.84 ms
[2025-10-08 20:02:52][info][yolo.cpp:304]:Engine destroy.
[100%] Built target yolo
```

fps从350提升至360.

3.5.2 Int8量化

INT8量化校准核心原理：

INT8量化（整数8位量化）通过将32位浮点数模型参数压缩为8位整数，可减少75%内存占用并提升2-4倍推理速度，是边缘设备部署的关键技术。



校准过程本质是通过代表性数据计算激活值的动态范围，生成缩放因子（scale）和零偏移（zero point）。

校准数据集构建：

1. 数据量与多样性平衡

Ultralytics源码明确建议校准集规模应不少于300张图像：

```
# 来自ultralytics/engine/exporter.py第590行
LOGGER.warning(f"{prefix} >300 images recommended for INT8 calibration, found {n} images.")
```

最佳实践：

- 从训练集中随机抽取300-500张图像
- 确保覆盖所有类别，小目标样本比例不低于15%
- 包含不同光照（白天/夜晚）、天气（晴/雨/雾）场景

本项目直接选择全部的验证集数据600张作为校准数据集，修改`src/application/app_yolo.cpp`文件第126行，设置`int8ImageDirectory`路径：

```
    if(not iLogger::exists(model_file)){
        TRT::compile(
            mode,                       // FP32、FP16、INT8
            test_batch_size,            // max batch size
            onnx_file,                  // source
            model_file,                 // save to
            {},
            int8process,
            "/data/coding/yolo_data/val/images"//校准数据集路径
        );
    }
```

2. 数据预处理陷阱规避

量化校准的数据预处理必须与推理阶段完全一致。


```
void image_to_tensor(const cv::Mat& image, shared_ptr<TRT::Tensor>& tensor, Type type, int ibatch){
  CUDAKernel::Norm normalize;
  if(type == Type::V5 || type == Type::V3 || type == Type::V7 || type == Type::V8 || type == Type::V6 || type == Type::V9 ||
     type == Type::V10 || type == Type::V11 || type == Type::V12 || type == Type::V13){
```

```
    normalize = CUDAKernel::Norm::alpha_beta(1 / 255.0f, 0.0f, CUDAKernel::ChannelType::Invert);
  }else if(type == Type::X){
    //float mean[] = {0.485, 0.456, 0.406};
    //float std[]  = {0.229, 0.224, 0.225};
    //normalize_ = CUDAKernel::Norm::mean_std(mean, std, 1/255.0f, CUDAKernel::ChannelType::Invert);
    normalize = CUDAKernel::Norm::None();
  }else{
    INFOE("Unsupport type %d", type);
  }

  Size input_size(tensor->size(3), tensor->size(2));
  AffineMatrix affine;
  affine.compute(image.size(), input_size);

  size_t size_image      = image.cols * image.rows * 3;
  size_t size_matrix     = iLogger::upbound(sizeof(affine.d2i), 32);
  auto workspace         = tensor->get_workspace();
  uint8_t* gpu_workspace        = (uint8_t*)workspace->gpu(size_matrix + size_image);
  float*   affine_matrix_device = (float*)gpu_workspace;
  uint8_t* image_device         = size_matrix + gpu_workspace;

  uint8_t* cpu_workspace        = (uint8_t*)workspace->cpu(size_matrix + size_image);
  float* affine_matrix_host     = (float*)cpu_workspace;
  uint8_t* image_host           = size_matrix + cpu_workspace;
  auto stream                   = tensor->get_stream();

  memcpy(image_host, image.data, size_image);
  memcpy(affine_matrix_host, affine.d2i, sizeof(affine.d2i));
  checkCudaRuntime(cudaMemcpyAsync(image_device, image_host, size_image, cudaMemcpyHostToDevice, stream));
  checkCudaRuntime(cudaMemcpyAsync(affine_matrix_device, affine_matrix_host, sizeof(affine.d2i), cudaMemcpyHostToDevice, stream));

  CUDAKernel::warp_affine_bilinear_and_normalize_plane(
    image_device,               image.cols * 3,       image.cols, image.rows,
    tensor->gpu<float>(ibatch), input_size.width,     input_size.height,
    affine_matrix_device, 114,
    normalize, stream
  );
  tensor->synchronize();
}
```

3. 主程序设置

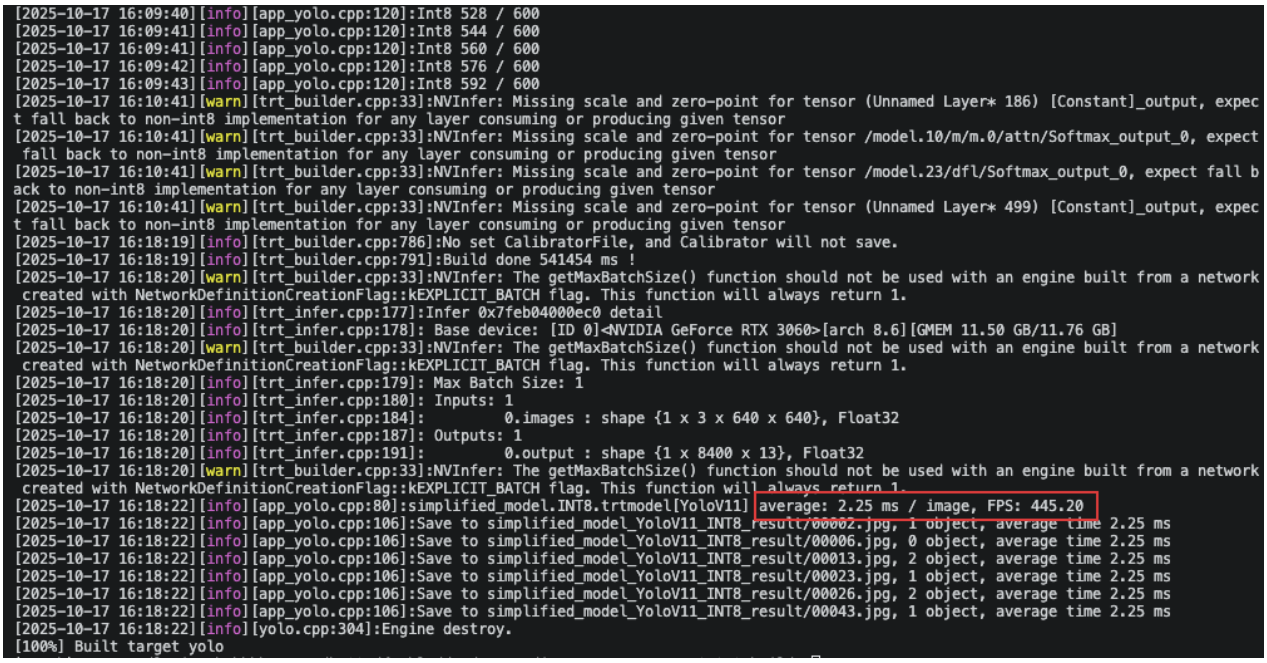
在文件`src/application/app_yolo.cpp`中修改，将`Mode`设置为`INT8`：

```cpp
int app_yolo(){

    // test(Yolo::Type::V8, TRT::Mode::FP32, "best");
    // test_single_image();
    // test_video();
    // test(Yolo::Type::V3, TRT::Mode::FP32, "yolov3");
    // test(Yolo::Type::X, TRT::Mode::FP32, "yolox_s");
    // test(Yolo::Type::V5, TRT::Mode::FP32, "yolov5s");
    // test(Yolo::Type::V6, TRT::Mode::FP32, "yolov6s");
    // test(Yolo::Type::V7, TRT::Mode::FP32, "yolov7");
    // test(Yolo::Type::V9, TRT::Mode::FP32, "yolov9c");
    // test(Yolo::Type::V10, TRT::Mode::FP32, "yolov10s");
    test(Yolo::Type::V11, TRT::Mode::INT8, "simplified_model");
    // test(Yolo::Type::V12, TRT::Mode::FP32, "yolov12s");
    // test(Yolo::Type::V13, TRT::Mode::FP32, "yolov13s");
    return 0;
}
```

4. 重新编译

```bash
cd build
make yolo -j32
```

```
(torch) root@evdltginocbohbhb-snow-db4885df5-bltdd:/data/coding/tensorRT_Pro-YOLOv8/build# make yolo -j32
[ 21%] Built target plugin_list
[ 22%] Building CXX object CMakeFiles/pro.dir/src/application/app_yolo.cpp.o
[ 23%] Linking CXX executable /data/coding/tensorRT_Pro-YOLOv8/workspace/pro
[100%] Built target pro
[2025-10-17 16:09:08][info][app_yolo.cpp:129]:==================== test YoloV11 INT8 simplified_model ====================================
[2025-10-17 16:09:08][info][trt_builder.cpp:565]:Compile INT8 Onnx Model 'simplified_model.onnx'.
[2025-10-17 16:09:18][warn][trt_builder.cpp:33]:NVInfer: /data/coding/tensorRT_Pro-YOLOv8/src/tensorRT/onnx_parser/onnx2trt_utils.cpp:372: You
r ONNX model has been generated with INT64 weights, while TensorRT does not natively support INT64. Attempting to cast down to INT32.
[2025-10-17 16:09:18][info][trt_builder.cpp:632]:Using entropy calibrator
[2025-10-17 16:09:18][info][trt_builder.cpp:643]:Using image list[600 files]: /data/coding/yolo_data/val/images
[2025-10-17 16:09:18][info][trt_builder.cpp:672]:Input shape is -1 x 3 x 640 x 640
[2025-10-17 16:09:18][info][trt_builder.cpp:673]:Set max batch size = 16
[2025-10-17 16:09:18][info][trt_builder.cpp:674]:Set max workspace size = 1024.00 MB
[2025-10-17 16:09:18][info][trt_builder.cpp:675]:Base device: [ID 0]<NVIDIA GeForce RTX 3060>[arch 8.6][GMEM 11.39 GB/11.76 GB]
[2025-10-17 16:09:18][info][trt_builder.cpp:678]:Network has 1 inputs:
[2025-10-17 16:09:18][info][trt_builder.cpp:684]:      0.[images] shape is -1 x 3 x 640 x 640
[2025-10-17 16:09:18][info][trt_builder.cpp:690]:Network has 1 outputs:
[2025-10-17 16:09:18][info][trt_builder.cpp:695]:      0.[output] shape is -1 x 8400 x 13
[2025-10-17 16:09:18][info][trt_builder.cpp:699]:Network has 510 layers:
[2025-10-17 16:09:18][info][trt_builder.cpp:766]:Building engine...
[2025-10-17 16:09:18][warn][trt_builder.cpp:33]:NVInfer: Calibration Profile is not defined. Calibrating with Profile 0
[2025-10-17 16:09:21][info][app_yolo.cpp:120]:Int8 16 / 600
[2025-10-17 16:09:22][info][app_yolo.cpp:120]:Int8 32 / 600
[2025-10-17 16:09:22][info][app_yolo.cpp:120]:Int8 48 / 600
[2025-10-17 16:09:23][info][app_yolo.cpp:120]:Int8 64 / 600
[2025-10-17 16:09:24][info][app_yolo.cpp:120]:Int8 80 / 600
[2025-10-17 16:09:24][info][app_yolo.cpp:120]:Int8 96 / 600
[2025-10-17 16:09:25][info][app_yolo.cpp:120]:Int8 112 / 600
[2025-10-17 16:09:25][info][app_yolo.cpp:120]:Int8 128 / 600
[2025-10-17 16:09:26][info][app_yolo.cpp:120]:Int8 144 / 600
[2025-10-17 16:09:27][info][app_yolo.cpp:120]:Int8 160 / 600
[2025-10-17 16:09:27][info][app_yolo.cpp:120]:Int8 176 / 600
```



可以看到，经过量化后：

- 推理速度提升，为FP32的1.39倍。虽然有提升，但是很不理想。理论上int8推理速度为fp32的3-4倍。正常情况下实际加速2.5倍。
- 推理精度下降较多

5. 推理精度评估

- 将 yolo 格式的标签转换成 json 文件, 用于 mAP 的计算

修改tools/yolo2json.py文件如下:

```python
if __name__ == "__main__":

    """
    1. 该脚本文件用于将 yolo 格式的标签转换成 json 文件, 用于 mAP 的计算
    2. 生成的 json 文件保存在 workspace/val.json
    """

    # Define the paths
    data_path   = "/data/coding/yolo_data/val/images"
    label_path  = "/data/coding/yolo_data/val/labels"

    # 类别名称请务必与 YOLO 格式的标签对应
    class_names = [
        "Crack",
        "Manhole",
        "Net",
        "Pothole",
        "Patch-Crack",
        "Patch-Net",
        "Patch-Pothole",
        "other",
        "Other"
```

```
        ]
    save_path   = "workspace/val.json"

    img_info = get_img_info(data_path, label_path)
    generate_coco_format_labels(img_info, class_names, save_path)
```

执行：

```
python tools/yolo2json.py
```

- 生成预测标签

修改src/application/test_yolo_map.cpp文件代码如下：

```
auto images = scan_dataset("/data/coding/yolo_data/val/images");
INFO("images.size = %d", images.size());

string model = "simplified_model.INT8.trtmodel";
inference(images, 0, model, TRT::Mode::INT8, Yolo::Type::V6);
save_to_json(images, model + ".prediction.json");
return 0;
```

重新编译执行：

```
cd build
make test_yolo_map -j32
```

- 计算map

修改/data/coding/tensorRT_Pro-YOLOv8/tools/mAP_test.py文件如下：

```
    nc = 9
    names =  [
        "Crack",
        "Manhole",
        "Net",
        "Pothole",
        "Patch-Crack",
        "Patch-Net",
        "Patch-Pothole",
        "other",
        "Other"
        ]
    anno_json = "workspace/val.json"
    pred_json = "workspace/simplified_model.INT8.trtmodel.prediction.json"
    eval_model(nc, names, anno_json, pred_json)
```

执行：

```
python tools/mAP_test.py
```

得到如下结果：

```
Class           Labeled_images      Labels     P@.5iou     R@.5iou    F1@.5iou      mAP@.5  mAP@.5:.95
all                        600        1151        0.37        0.39        0.38        0.28       0.146
Crack                      130         205       0.373        0.35       0.361       0.231      0.0855
Manhole                    326         444       0.811        0.83       0.821       0.824       0.472
Net                         30          38       0.235        0.31       0.268       0.113      0.0537
Pothole                     18          22       0.235        0.18       0.204      0.0443      0.0191
Patch-Crack                137         249       0.521         0.4       0.452       0.363        0.14
Patch-Net                   55         105       0.376        0.39       0.383       0.242      0.0947
Patch-Pothole               66          80       0.561        0.46       0.505       0.419       0.302
/data/miniconda/envs/torch/lib/python3.10/site-packages/numpy/core/fromnumeric.py:3464: RuntimeWarning:
  return _methods._mean(a, axis=axis, dtype=dtype,
/data/miniconda/envs/torch/lib/python3.10/site-packages/numpy/core/_methods.py:192: RuntimeWarning: inv
  ret = ret.dtype.type(ret / rcount)
other                       0           0          -1        0.99         198         nan         nan
Other                       7           8           0           0           0           0           0
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.146
```

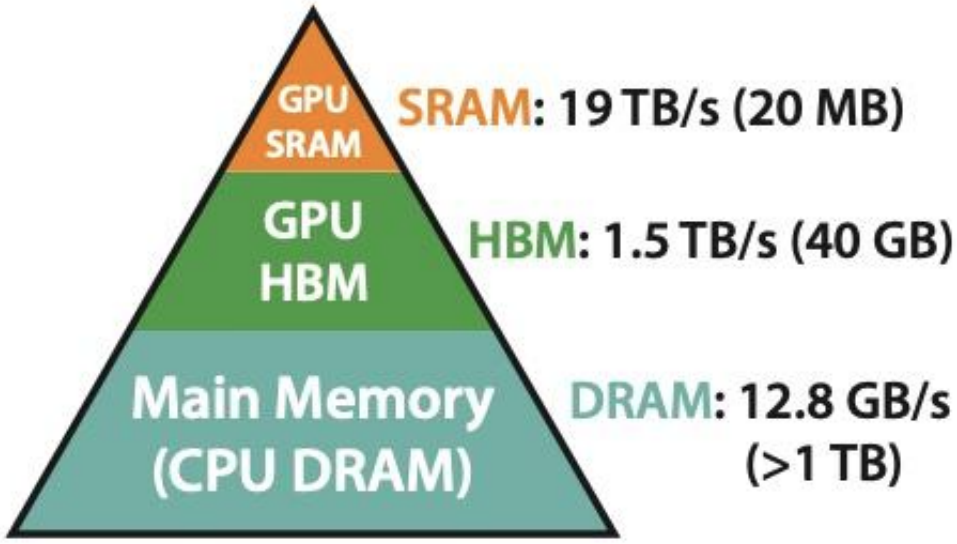
同理，可以计算fp32模型的性能如下：

```
 Class           Labeled_images      Labels     P@.5iou     R@.5iou    F1@.5iou      mAP@.5  mAP@.5:.95
 all                        600        1151       0.397        0.42       0.408       0.337       0.172
 Crack                      130         205       0.409        0.42       0.415       0.326       0.106
 Manhole                    326         444       0.828        0.86       0.843       0.883       0.531
 Net                         30          38        0.21        0.34       0.259       0.109      0.0418
 Pothole                     18          22         0.8        0.18       0.294       0.188      0.0565
 Patch-Crack                137         249       0.498        0.53       0.514       0.406       0.175
 Patch-Net                   55         105       0.396        0.36       0.377       0.281       0.124
 Patch-Pothole               66          80       0.661        0.48       0.556       0.504        0.34
 /data/miniconda/envs/torch/lib/python3.10/site-packages/numpy/core/fromnumeric.py:3464: RuntimeWarning:
   return _methods._mean(a, axis=axis, dtype=dtype,
 /data/miniconda/envs/torch/lib/python3.10/site-packages/numpy/core/_methods.py:192: RuntimeWarning: inv
   ret = ret.dtype.type(ret / rcount)
 other                       0           0          -1        0.99         198         nan         nan
 Other                       7           8           0           0           0           0           0
```

3.5.3 算子融合

将模型推理的解码部分手动融合为一个算子，可进一步提高模型推理性能。其原理为：

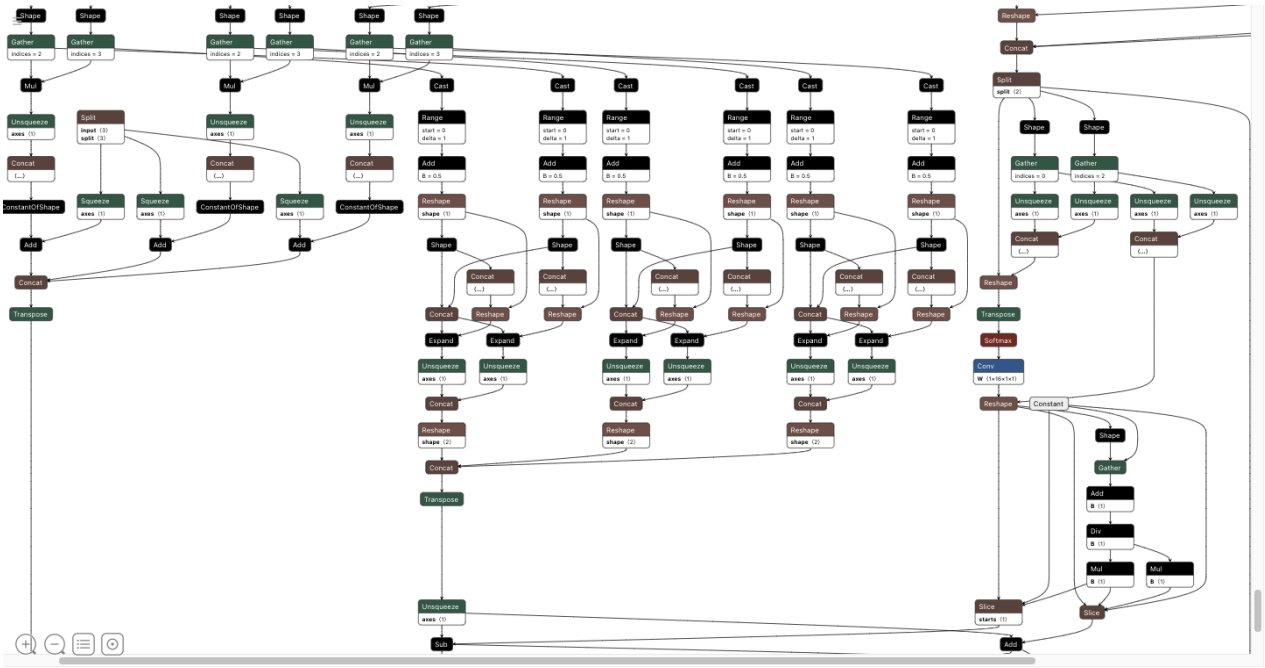
- 将模型推理中不利于量化的解码部分从tensorrt剥离，量化效果更好
- 多个算子融合，能够减少核函数的数量，从而提高性能



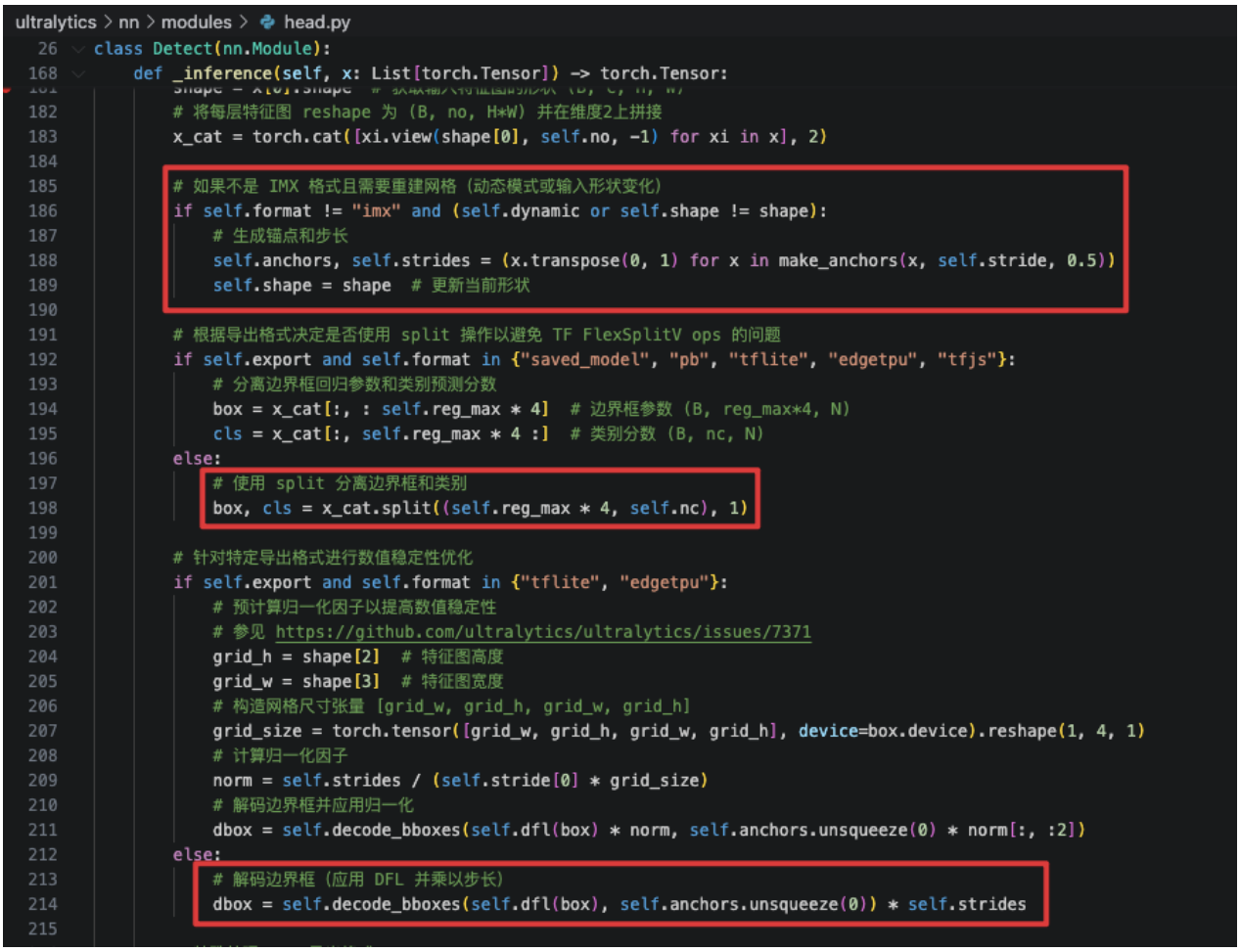
Memory Hierarchy with Bandwidth & Memory Size

3.5.3.1 分析解码细节

观察onnx计算图的输出部分：



对应代码：



> 这里存在make_anchors、split、conv、softmax、sigmoid、add、mulitiply等大量操作，这些操作完全可以整合到decode_kernel_v8这个后处理的解码核函数中。

```
f _inference(self, x: List[torch.Tensor]) -> torch.Tensor:
  """
  Decode predicted bounding boxes and class probabilities based on
le-level feature maps.

  推理阶段解码边界框和类别概率。

  Args:
      x (List[torch.Tensor]): List of feature maps from different
ion layers. 输入特征图列表
      0:1,73,80,80  1:1,73,40,40  2:1,73,20,20
  Returns:
      (torch.Tensor): Concatenated tensor of decoded bounding boxes
ass probabilities. 拼接后的预测张量
  """
  # 推理路径：将多层特征图拼接并解码为最终检测结果
  shape = x[0].shape  # 获取输入特征图的形状 (B, C, H, W)  1,73,80,80
  # 将每层特征图 reshape 为 (B, no, H*W) 并在维度2上拼接
  x_cat = torch.cat([xi.view(shape[0], self.no, -1) for xi in x],

  1,73,8400
  # 如果不是 IMX 格式且需要重建网格（动态模式或输入形状变化）
```

```
  if self.format != "imx" and (self.dynamic or self.shape !=
:
      # 生成锚点和步长  self.stride 8,16,32
      self.anchors, self.strides = (x.transpose(0, 1) for x in
nchors(x, self.stride, 0.5))
      self.shape = shape  # 更新当前形状

  # 根据导出格式决定是否使用 split 操作以避免 TF FlexSplitV ops 的问题
  if self.export and self.format in {"saved_model", "pb", "tflite",
pu", "tfjs"}:
      # 分离边界框回归参数和类别预测分数
      box = x_cat[:, : self.reg_max * 4]  # 边界框参数 (B,
x*4, N)
      cls = x_cat[:, self.reg_max * 4 :]  # 类别分数 (B, nc, N)
  else:
      # 使用 split 分离边界框和类别
      box, cls = x_cat.split((self.reg_max * 4, self.nc), 1)

  # 针对特定导出格式进行数值稳定性优化
  if self.export and self.format in {"tflite", "edgetpu"}:
      # 预计算归一化因子以提高数值稳定性
      # 参见 https://github.com/ultralytics/ultralytics/issues/7371
      grid_h = shape[2]  # 特征图高度
      grid_w = shape[3]  # 特征图宽度
      # 构造网格尺寸张量 [grid_w, grid_h, grid_w, grid_h]
      grid_size = torch.tensor([grid_w, grid_h, grid_w, grid_h],
=box.device).reshape(1, 4, 1)
      # 计算归一化因子
      norm = self.strides / (self.stride[0] * grid_size)
      # 解码边界框并应用归一化
      dbox = self.decode_bboxes(self.dfl(box) * norm,
nchors.unsqueeze(0) * norm[:, :2])
  else:
      # 解码边界框（应用 DFL 并乘以步长）
      dbox = self.decode_bboxes(self.dfl(box),
nchors.unsqueeze(0)) * self.strides

  # 特殊处理 IMX 导出格式
  if self.export and self.format == "imx":
      # 调整输出维度顺序并返回边界框和类别分数
      return dbox.transpose(1, 2), cls.sigmoid().permute(0, 2, 1)
  # 返回拼接后的边界框和类别分数（类别分数经过 sigmoid 激活）
  return torch.cat((dbox, cls.sigmoid()), 1)
f decode_bboxes(self, bboxes: torch.Tensor, anchors: torch.Tensor,
bool = True) -> torch.Tensor:
  """
  Decode bounding boxes from predictions.

  从预测结果解码边界框。
```

```
    """
    return dist2bbox(bboxes, anchors, xywh=xywh and not (self.end2end
f.xyxy), dim=1)


DFL(nn.Module):
"
tegral module of Distribution Focal Loss (DFL).

布式焦点损失（DFL）积分模块。
用于目标检测框回归的分布式预测。
"

f __init__(self, c1: int = 16):
    """
    Initialize a convolutional layer with a given number of input
ls.

    初始化DFL模块，构建一个特殊的1x1卷积用于分布式回归。

    Args:
        c1 (int): Number of input channels. 输入通道数
    """
    super().__init__()
    self.conv = nn.Conv2d(c1, 1, 1, bias=False).requires_grad_(False)
训练的1x1卷积
    x = torch.arange(c1, dtype=torch.float)
    self.conv.weight.data[:] = nn.Parameter(x.view(1, c1, 1, 1))  #
始化为0~c1-1
    self.c1 = c1

f forward(self, x: torch.Tensor) -> torch.Tensor:
    """
    Apply the DFL module to input tensor and return transformed
.

    前向传播：对输入张量进行分布式回归积分。

    Args:
        x (torch.Tensor): 输入特征张量
    Returns:
        (torch.Tensor): 回归后的输出张量
    """
    b, _, a = x.shape  # batch, channels, anchors
    # 先reshape再softmax，最后用不可训练卷积做加权平均
    return self.conv(x.view(b, 4, self.c1, a).transpose(2,
tmax(1)).view(b, 4, a)
    # return self.conv(x.view(b, self.c1, 4, a).softmax(1)).view(b,
```

```
st2bbox(distance, anchor_points, xywh=True, dim=-1):
"
ansform distance(ltrb) to box(xywh or xyxy).
边界框的相对距离(ltrb)转换为绝对坐标(xywh或xyxy格式)

gs:
  distance (torch.Tensor): 边界框的偏移量，格式为[l,t,r,b]（左、上、右、
., 4, 8400]
  anchor_points (torch.Tensor): 锚点坐标（通常是边界框的中心点）
  xywh (bool, optional): 输出格式开关。True=中心点+宽高(xywh)，False=角
xyxy)。默认True
  dim (int, optional): 操作维度。默认-1（最后一个维度）

turns:
  torch.Tensor: 转换后的边界框坐标（根据xywh参数返回xywh或xyxy格式）
"
将距离张量分割为左上(lt)和右下(rb)两部分
distance.chunk(2, dim) -> [lt, rb] 其中lt=[l,t], rb=[r,b]
, rb = distance.chunk(2, dim)
计算边界框的左上角坐标：锚点 - 左上偏移量
y1 = anchor_points - lt
计算边界框的右下角坐标：锚点 + 右下偏移量
y2 = anchor_points + rb
根据输出格式要求返回不同结果
 xywh:
  # 计算中心点坐标：(左上+右下)/2
  c_xy = (x1y1 + x2y2) / 2
  # 计算宽高：右下 - 左上
  wh = x2y2 - x1y1
  # 拼接中心点坐标和宽高 -> [x_center, y_center, width, height]
  return torch.cat((c_xy, wh), dim)  # xywh bbox
直接拼接左上和右下坐标 -> [x1, y1, x2, y2]
turn torch.cat((x1y1, x2y2), dim)  # xyxy bbox
ke_anchors(feats, strides, grid_cell_offset=0.5):
"Generate anchors from features."""
chor_points, stride_tensor = [], []
sert feats is not None
ype, device = feats[0].dtype, feats[0].device
r i, stride in enumerate(strides):
  h, w = feats[i].shape[2:] if isinstance(feats, list) else
eats[i][0]), int(feats[i][1]))
  sx = torch.arange(end=w, device=device, dtype=dtype) +
ell_offset  # shift x
  sy = torch.arange(end=h, device=device, dtype=dtype) +
ell_offset  # shift y
  sy, sx = torch.meshgrid(sy, sx, indexing="ij") if TORCH_1_10 else
torch.meshgrid(sy, sx)
  anchor_points.append(torch.stack((sx, sy), -1).view(-1, 2))
```

```
144     stride_tensor.append(torch.full((h * w, 1), stride, dtype=dtype,
    device=device))
145 return torch.cat(anchor_points), torch.cat(stride_tensor)
```

3.5.3.2 重新导出onnx

进入项目`ultralytics`，修改`ultralytics/nn/modules/head.py`文件，修改第143行如下：

```
        if self.export:
            shapes = [xi.shape[2:] for xi in x]  # 获取每层特征图的h和w
            shape = x[0].shape
            x_cat = torch.cat([xi.view(-1, self.no, shape[0]*shape[1]) for
xi, shape in zip(x, shapes)], 2)
            return x_cat.permute(0, 2, 1)
        return (y, x)
```

```
for i in range(self.nl):
    x[i] = torch.cat((self.cv2[i](x[i]), self.cv3[i](x[i])), 1)
if self.training:  # Training path
    return x
y: Tensor = self._inference(x)

# return y if self.export else (y, x)
# return y.permute(0, 2, 1) if self.export else (y, x)# (-1,13,8400)-->(-1,8400,13)
if self.export:
    shapes: list[Size] = [xi.shape[2:] for xi in x]  # 获取每层特征图的h和w
    shape: Size = x[0].shape
    x_cat: Tensor = torch.cat([xi.view(-1, self.no, shape[0]*shape[1]) for xi, shape in zip[tuple[Tensor, Size]](x, shapes)
    return x_cat.permute(0, 2, 1)
return (y, x)
```

在终端执行如下指令即可完成 onnx 导出：

```
python export.py
```

3.5.3.3 编写核函数

进入项目`tensorRT_Pro-YOLOv8`，在`src/application/app_yolo/yolo_decode.cu`文件中新增如下核函数：

```
/**
     * @brief CUDA核函数，用于YOLOv8模型的边界框解码
     *
     * 该函数负责将YOLOv8模型的预测输出转换为图像坐标系中的边界框，
     * 并根据置信度阈值过滤预测结果。
     *
     * @param predict 指向模型预测结果的指针，格式为 [1,num_boxes,73]
     * @param num_bboxes 预测的边界框总数
     * @param num_classes 分类数量
     * @param confidence_threshold 置信度阈值，低于此值的预测将被过滤
     * @param invert_affine_matrix 逆仿射变换矩阵，用于将坐标转换回原始图像空间
     * @param parray 输出数组，用于存储解码后的边界框信息
     * @param MAX_IMAGE_BOXES 每张图像允许的最大边界框数量
     */
    static __global__ void decode_kernel_v8_with_decode(
```

```
        float *predict,
        int num_bboxes,
        int num_classes,
        float confidence_threshold,
        float* invert_affine_matrix,
        float* parray,
        int MAX_IMAGE_BOXES
    ){

        // 计算当前线程处理的边界框索引
        int position = blockDim.x * blockIdx.x + threadIdx.x;

        // 边界检查，超出范围直接返回
        if (position >= num_bboxes) return;
        // 定位到当前预测数据的起始位置
        float* pred_ptr = predict + (64 + num_classes) * position;
        // 指向类别置信度数据的起始位置
        float* class_confidence = pred_ptr + 64;

        // 初始化最高类别置信度和对应标签
        float confidence = *class_confidence++;  // 第0类的置信度
        int label = 0;

        // 遍历所有类别，找到置信度最高的类别
        for(int i = 1; i < num_classes; ++i, ++class_confidence){
            if(*class_confidence > confidence){
                confidence = *class_confidence;
                label = i;
            }
        }
        //计算置信度的sigmoid
        confidence = 1.0f / (1.0f + expf(-confidence));


        // 如果最高类别置信度低于阈值，则过滤该预测
        if(confidence < confidence_threshold)
            return;

        // 原子操作获取输出数组中的索引位置
        int index = atomicAdd(parray, 1);
        // 如果超出最大边界框数量限制，则丢弃
        if(index >= MAX_IMAGE_BOXES)
            return;

        // 计算在特征图上的二维坐标
        int grid_y, grid_x,stride;
        if (position<6400) {
            stride = 8;
            grid_y=position/80;
```

```
            grid_x=position%80;
        }
        else if(position<8000){
            stride = 16;
            grid_y=(position-6400)/40;
            grid_x=(position-6400)%40;
        }else{
            stride = 32;
            grid_y=(position-8000) / 20;
            grid_x=(position-8000) % 20;
        }

        float points[4]={0.0f}; //存储边界框的坐标信息
        // 处理前64个元素，分为4组，每组16个元素计算softmax和点积
        #pragma unroll
        for (int group = 0; group < 4; ++group) {
            float max_val = pred_ptr[group * 16];
            // 寻找最大值用于数值稳定
            #pragma unroll
            for (int i = 1; i < 16; ++i) {
                if (pred_ptr[group * 16 + i] > max_val) {
                    max_val = pred_ptr[group * 16 + i];
                }
            }

            // 计算softmax并点积得到距离值
            float sum_exp = 0.0f;
            #pragma unroll
            for (int i = 0; i < 16; ++i) {
                sum_exp += expf(pred_ptr[group * 16 + i] - max_val);
            }

            // 计算加权平均得到距离值（这里假设权重为 1~16）
            #pragma unroll
            for (int i = 0; i < 16; ++i) {
                float prob = expf(pred_ptr[group * 16 + i] - max_val) /
sum_exp;
                points[group] += prob * (i + 1);  // 权重为 1 到 16
            }
        }

        points[0] = (grid_x+0.5-points[0])*stride;
        points[1] = (grid_y+0.5-points[1])*stride;
        points[2] = (grid_x+0.5+points[2])*stride;
        points[3] = (grid_y+0.5+points[3])*stride;

        // 应用逆仿射变换，将坐标转换回原始图像空间
        affine_project(invert_affine_matrix, points[0], points[1],
&points[0],  &points[1]);
```

```
        affine_project(invert_affine_matrix, points[2], points[3],
&points[2], &points[3]);

        // 计算输出数组中当前边界框的存储位置
        // parray[0]存储边界框计数，实际数据从parray[1]开始
        float *pout_item = parray + 1 + index * NUM_BOX_ELEMENT;

        // 存储边界框信息：left, top, right, bottom, confidence, label,
keepflag
        *pout_item++ = points[0];      // 左边界
        *pout_item++ = points[1];       // 上边界
        *pout_item++ = points[2];     // 右边界
        *pout_item++ = points[3];    // 下边界
        *pout_item++ = confidence; // 置信度
        *pout_item++ = label;     // 类别标签
        *pout_item++ = 1;         // 保留标志：1=保留，0=忽略
    }
```

3.5.3.4 修改代码

- 修改`src/application/app_yolo/yolo.cpp`文件第30行，新增一个type，便于对算子融合版本进行特定处理：

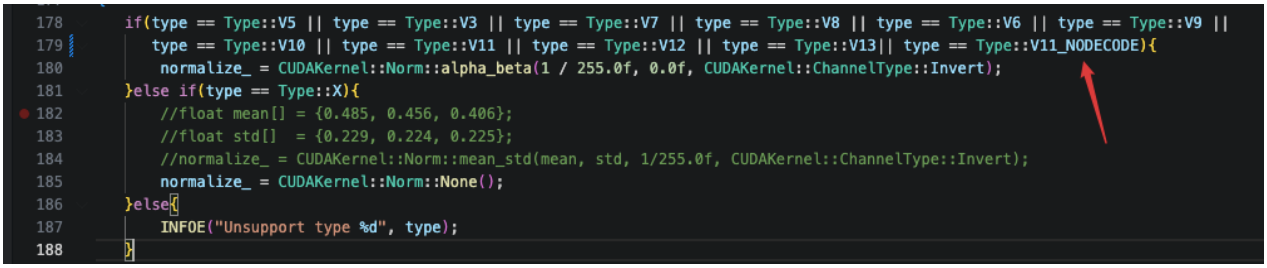
```
    const char* type_name(Type type){
        switch(type){
        case Type::V5: return "YoloV5";
        case Type::V3: return "YoloV3";
        case Type::V7: return "YoloV7";
        case Type::V6: return "YoloV6";
        case Type::V8: return "YoloV8";
        case Type::X: return "YoloX";
        case Type::V9: return "YoloV9";
        case Type::V10: return "YoloV10";
        case Type::V11: return "YoloV11";
        case Type::V12: return "YoloV12";
        case Type::V13: return "YoloV13";
        case Type::V11_NODECODE: return "YoloV11_nodecode";//新增一个type
        default: return "Unknow";
        }
    }
```

- 修改头文件`src/application/app_yolo/yolo.hpp`第33行，新增对应的`Type`：

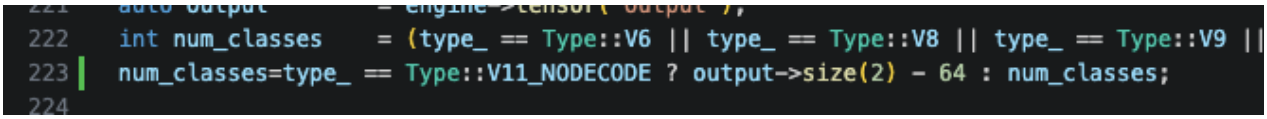
```
    enum class Type : int{
        V5  = 0,
        X   = 1,
        V3  = 2,
        V7  = 3,
```

```
        V8  = 4,
        V6  = 5,
        V9  = 6,
        V10 = 7,
        V11 = 8,
        V12 = 9,
        V13 = 10,
        V11_NODECODE = 11
    };
```

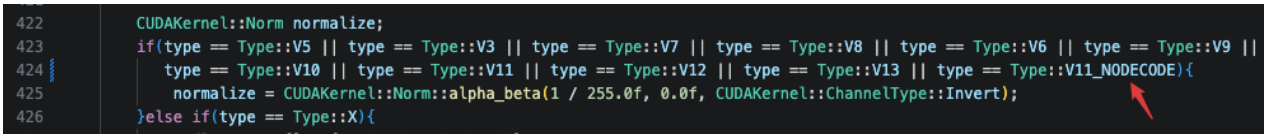
- 修改`src/application/app_yolo/yolo.cpp`文件第178,221,421行，新增对`Type::V11_NODECODE`的判断：



```
if(type == Type::V5 || type == Type::V3 || type == Type::V7 || type == Type::V8 || type == Type::V6 || type == Type::V9 ||
   type == Type::V10 || type == Type::V11 || type == Type::V12 || type == Type::V13|| type == Type::V11_NODECODE){
    normalize_ = CUDAKernel::Norm::alpha_beta(1 / 255.0f, 0.0f, CUDAKernel::ChannelType::Invert);
}else if(type == Type::X){
    //float mean[] = {0.485, 0.456, 0.406};
    //float std[]  = {0.229, 0.224, 0.225};
    //normalize_ = CUDAKernel::Norm::mean_std(mean, std, 1/255.0f, CUDAKernel::ChannelType::Invert);
    normalize_ = CUDAKernel::Norm::None();
}else{
    INFOE("Unsupport type %d", type);
}
```



```
int num_classes    = (type_ == Type::V6 || type_ == Type::V8 || type_ == Type::V9 ||
num_classes=type_ == Type::V11_NODECODE ? output->size(2) - 64 : num_classes;
```



```
CUDAKernel::Norm normalize;
if(type == Type::V5 || type == Type::V3 || type == Type::V7 || type == Type::V8 || type == Type::V6 || type == Type::V9 ||
   type == Type::V10 || type == Type::V11 || type == Type::V12 || type == Type::V13 || type == Type::V11_NODECODE){
    normalize = CUDAKernel::Norm::alpha_beta(1 / 255.0f, 0.0f, CUDAKernel::ChannelType::Invert);
}else if(type == Type::X){
```

- 修改`src/application/app_yolo/yolo_decode.cu`文件中的`decode_kernel_invoker`函数，增加对`Type::V11_NODECODE`的判断和对应的核函数处理：

```
    void decode_kernel_invoker(float* predict, int num_bboxes, int num_classes, float confidence_threshold, float* invert_affine_matrix, float* parray, int max_objects, cudaStream_t stream, Type type){

        auto grid = CUDATools::grid_dims(num_bboxes);
        auto block = CUDATools::block_dims(num_bboxes);
        if(type == Type::V6 || type == Type::V8 || type == Type::V9 || type == Type::V11 || type == Type::V12 || type == Type::V13){
            checkCudaKernel(decode_kernel_v8<<<grid, block, 0, stream>>>(predict, num_bboxes, num_classes, confidence_threshold, invert_affine_matrix, parray, max_objects));
        }else if(type == Type::V11_NODECODE){//此种类型时，采用新增的核函数进行处理
            checkCudaKernel(decode_kernel_v8_with_decode<<<grid, block, 0, stream>>>(predict, num_bboxes, num_classes, confidence_threshold, invert_affine_matrix, parray, max_objects));
        }else if(type == Type::V10){
```

```
            checkCudaKernel(decode_kernel_v10<<<grid, block, 0, stream>>>(predict, num_bboxes, confidence_threshold, invert_affine_matrix, parray, max_objects))
        }else{
            checkCudaKernel(decode_kernel<<<grid, block, 0, stream>>>(predict, num_bboxes, num_classes, confidence_threshold, invert_affine_matrix, parray, max_objects));
        }
    }
```

3.5.3.5 编译运行

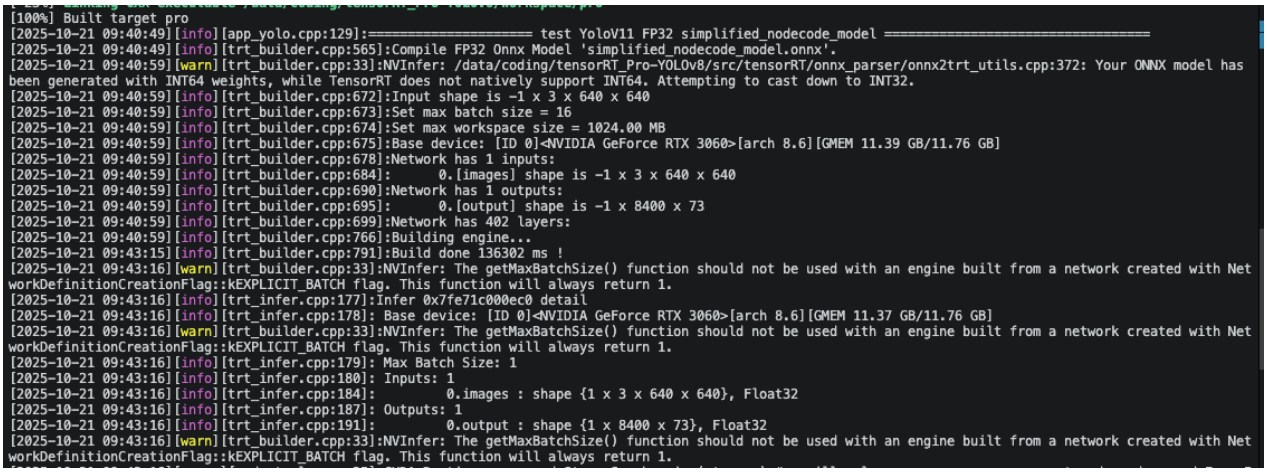
将刚刚导出的新onnx文件重命名后上传到workspace目录下，然后在`src/application/app_yolo.cpp`文件中左如下修改：

```
int app_yolo(){

    // test(Yolo::Type::V8, TRT::Mode::FP32, "best");
    // test_single_image();
    // test_video();
    // test(Yolo::Type::V3, TRT::Mode::FP32, "yolov3");
    // test(Yolo::Type::X, TRT::Mode::FP32, "yolox_s");
    // test(Yolo::Type::V5, TRT::Mode::FP32, "yolov5s");
    // test(Yolo::Type::V6, TRT::Mode::FP32, "yolov6s");
    // test(Yolo::Type::V7, TRT::Mode::FP32, "yolov7");
    // test(Yolo::Type::V9, TRT::Mode::FP32, "yolov9c");
    // test(Yolo::Type::V10, TRT::Mode::FP32, "yolov10s");
    test(Yolo::Type::V11, TRT::Mode::FP32, "simplified_model");
    test(Yolo::Type::V11, TRT::Mode::INT8, "simplified_model");
    test(Yolo::Type::V11_NODECODE, TRT::Mode::FP32, "simplified_nodecode_model");
    test(Yolo::Type::V11_NODECODE, TRT::Mode::INT8, "simplified_nodecode_model");
    // test(Yolo::Type::V12, TRT::Mode::FP32, "yolov12s");
    // test(Yolo::Type::V13, TRT::Mode::FP32, "yolov13s");
    return 0;
}
```

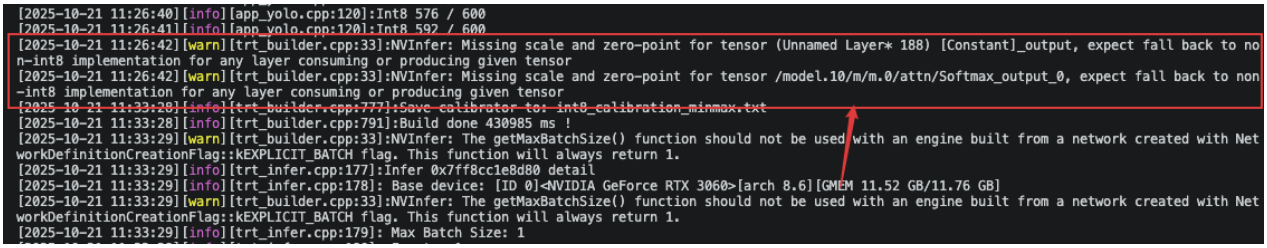
> 注意：onnx文件名请根据实际名称修改。

然后执行：

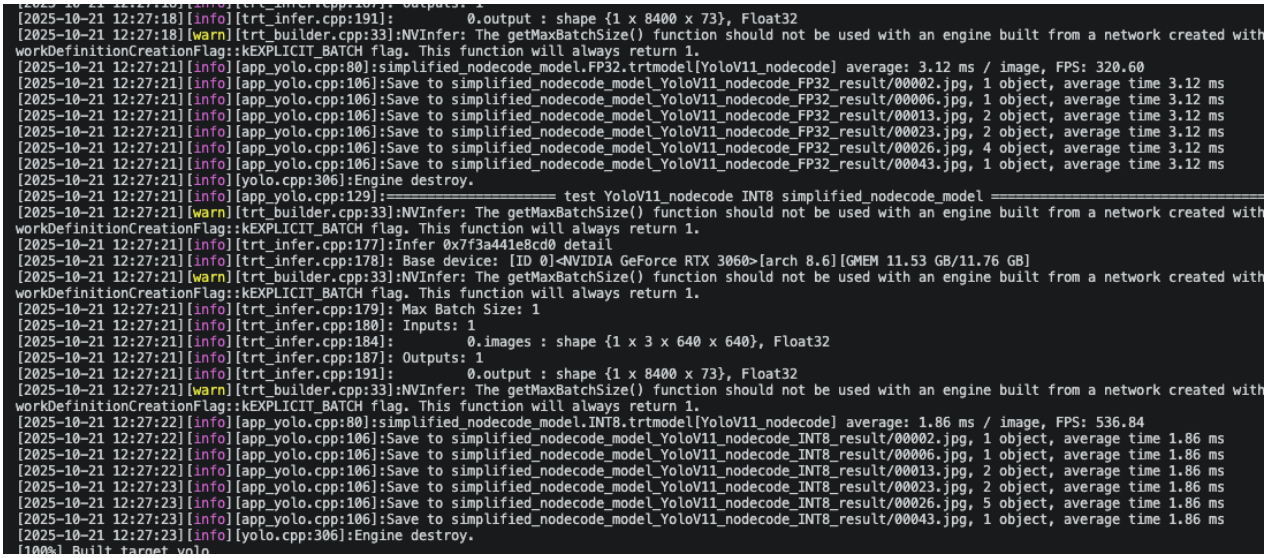
```
cd build
make yolo -j32
```



> 这个图中可以看到，拿掉解码部分后，计算图中的层数只有402层，tensorrt的模型推理部分会大大加速。



> 这个图中可以看到，相比融合前，不可量化的解码部分消失了。说明之前的量化版本因为解码部分存在太多没有量化实现的算子，导致存在过多量化和反量化过程。这将导致：
>
> - 推理速度减慢
> - 推理精度变差
>
> 这也是导致3.5.2量化结果非常差的原因。



> 这个图中可以看到，算子融合后，推理速度得到一定提升。更重要的是，Int8量化后精度损失较小。

4 triton server服务化部署